Linux for engineers

Navigate, secure, package, schedule and troubleshoot a Linux server the way production work demands.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. Filesystem and navigation (14 min)
2. Users, groups and permissions (15 min)
3. Processes and services with systemd (15 min)
4. Disks and filesystems (15 min)
5. LVM and software RAID (16 min)
6. Package management and repositories (14 min)
7. SSH and secure remote access (15 min)
8. Scheduling with cron and systemd timers (14 min)
9. Logs and troubleshooting (15 min)
10. Performance triage (16 min)
11. Boot process, kernel and recovery (15 min)
12. Server hardening and auditing (16 min)
13. Namespaces and cgroups: what a container really is (15 min)
14. Working efficiently in the terminal (12 min)
15. Time, NTP and time zones (12 min)
16. systemd in depth: overrides, targets and dependencies (14 min)
17. Kernel modules, sysctl and boot parameters (13 min)
18. Memory, swap and the OOM killer (14 min)
19. System calls, strace and perf (14 min)
20. Filesystems, the page cache and I/O (14 min)
21. Networking in the kernel: sockets, queues and conntrack (14 min)
22. Debugging a sick Linux server (16 min)
23. Troubleshooting playbook: method and the first five minutes (12 min)
24. Troubleshooting playbook: high load and CPU saturation (12 min)
25. Troubleshooting playbook: memory pressure, swap and the OOM killer (12 min)
26. Troubleshooting playbook: disk full, inodes and I/O stalls (12 min)
27. Troubleshooting playbook: network drops and slow connections on a host (13 min)
28. Troubleshooting playbook: boot failures and services that will not start (12 min)

29. Debugging with strace: watch system calls (13 min)
30. Seeing the network with ss and tcpdump (14 min)
31. Open files and /proc: find leaks and who holds what (13 min)
32. Finding CPU, memory and I/O hogs (14 min)
33. Kernel messages, the journal and core dumps (13 min)
34. eBPF and bpftrace: ask live questions of a running system (14 min)
35. Debugging a slow or hung server: a method (14 min)

Cheat sheet

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 1. Filesystem and navigation

14 minutes. Learn where things live on a Linux server, how to find them fast, and how symbolic links cause real deployment bugs.

It is 02:10 and an alert says the checkout service is serving an old version of its config. You SSH into the server. Before you change anything you need to answer three questions: where am I, what is here, and what changed? This lesson gives you the map and the commands to answer them in under a minute.

What you will learn

- The standard directories and what belongs in each
- How to move around, list and search without guessing
- How to copy, move and delete without destroying data
- How symbolic links work, and how a stale link breaks a deployment

One tree, many purposes

Linux has a single directory tree that starts at `/` (root). There are no drive letters. Disks, USB sticks and network shares are *mounted* into the tree at a directory you choose. Knowing the standard layout tells you where to look before you search.

Path	What lives there	You will touch it when
`/etc`	System-wide configuration files	Changing service settings
`/var/log`	Log files	Investigating a failure
`/var/lib`	Application state, such as databases	Checking data size or restoring
`/home`	One directory per user	Looking at SSH keys or shell history
`/usr/bin`, `/usr/sbin`	Installed programs	Checking which binary runs
`/opt`	Self-contained third-party or in-house apps	Finding deployed releases
`/tmp`	Temporary files, often cleared on reboot	Debugging scratch files
`/proc`, `/sys`	Live kernel and process information, not real files	Reading CPU, memory, process data
`/dev`	Device files such as disks and terminals	Working with disks

Why it matters: *Configuration is almost always under `/etc`, logs under `/var/log`, and application data under `/var/lib` or `/opt`. When a vendor puts things elsewhere, that is the exception worth noting.*

Move around and look

```
$ pwd
/home/deploy
$ cd /var/log
$ ls -lah
total 4.2M
drwxrwxr-x  9 root   syslog 4.0K Oct 24 00:00 .
drwxr-xr-x 14 root   root   4.0K Sep  2 11:20 ..
-rw-r-----  1 syslog adm    312K Oct 24 02:09 auth.log
-rw-r-----  1 syslog adm    1.1M Oct 24 02:10 syslog
drwxr-xr-x  2 root   root   4.0K Oct 23 06:25 nginx
$ cd -
/home/deploy
```

Read the `ls -l` columns left to right: file type and permissions, link count, owner, group, size, modified time, name. The first character is the type: `-` file, `d` directory, `l` symbolic link. `-h` prints human-readable sizes and `-a` shows hidden files, which start with a dot. `cd -` jumps back to the previous directory.

The output above is sample output. Names, sizes and dates on your machine will differ.

MARGA by EknathaLabs marga.eknathalabs.com

Find things

You will often know a file name but not its location, or the reverse.

```
$ find /etc -name "nginx.conf"
/etc/nginx/nginx.conf
$ find /var/log -type f -mtime -1 -size +1M
/var/log/syslog
$ grep -rn "listen 443" /etc/nginx
/etc/nginx/sites-enabled/shop.conf:4:    listen 443 ssl;
$ type -a python3
python3 is /usr/bin/python3
```

`find` walks the tree and filters by name, type, age or size. `grep -rn` searches *inside* files recursively and prints line numbers. `type -a` shows which executable will actually run when you type a command, which settles most "wrong version" arguments.

Copy, move and delete without regret

```
cp -a /etc/nginx /root/nginx.bak.2026-10-24   # -a keeps permissions, owners and links
mv app.conf app.conf.old                       # rename in place
rm -i notes.txt                                # ask before deleting
```

> **Warning:** `rm` *does not use a recycle bin.* `rm -rf` *with a typo or an empty variable, such as* `rm -rf "$DIR"/`*, can erase far more than you meant. Before deleting, run the same pattern with* `ls` *and read the list. Always make a copy before editing a production config.*

Links: one name pointing at another

A **symbolic link** (symlink) is a small file that holds a path. A **hard link** is a second name for the same data on disk.

```
$ ln -s /etc/nginx/sites-available/shop.conf /etc/nginx/sites-enabled/shop.conf
$ ls -l /etc/nginx/sites-enabled/
lrwxrwxrwx 1 root root 38 Oct 20 08:12 shop.conf -> /etc/nginx/sites-available/shop.conf
```

If you delete the target of a symlink, the link keeps pointing at nothing and is called *broken*. Hard links cannot cross filesystems, and deleting one name does not remove the data until every name is gone.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Filesystem and navigation fits together

Part	What it does	What goes wrong
One tree, many purposes	Linux has a single directory tree that starts at / (root).	Forgetting -n when replacing a symlink that points to a directory.
Move around and look	Read the ls -l columns left to right: file type and permissions, link count, owner, group, size, modified time, name.	Searching with find / on a busy server instead of a narrower directory.
Find things	You will often know a file name but not its location, or the reverse.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Copy, move and delete without regret	.	Editing a config in /etc without a backup, then forgetting what the original said.
Links: one name pointing at another	A symbolic link (symlink) is a small file that holds a path.	Hard links cannot cross filesystems, and deleting one name does not remove the data until every name is gone.

Flow: One tree, many purposes to Move around and look / Move around and look to Find things / Find things to Copy, move and delete without regret / Copy, move and delete without regret to Links: one name pointing at another

MARGA by EknathaLabs marga.eknathalabs.com

Incident: the new release is deployed but the old config is live

The deploy script finished without errors, yet the service still behaves like last week's build. Symlinks are a common cause because many teams point `current` at the active release.

> **Incident:** *A deployment tool creates* `/opt/shop/releases/<id>` *and then swaps a* `current` *link. The service reads* `/opt/shop/current/config.yml`*.*

```
$ ls -l /opt/shop
total 8
lrwxrwxrwx 1 deploy deploy   26 Oct 14 16:02 current -> releases/2026-10-14-1a2b3c
drwxr-xr-x 4 deploy deploy 4096 Oct 21 09:14 releases
$ ls /opt/shop/releases
2026-10-14-1a2b3c  2026-10-21-9f8e7d
```

The newest release, `2026-10-21-9f8e7d`, exists, but `current` still points at the older one. The link was never updated. Repoint it, then verify:

```
$ ln -sfn releases/2026-10-21-9f8e7d /opt/shop/current
$ readlink -f /opt/shop/current
/opt/shop/releases/2026-10-21-9f8e7d
```

`-s` makes a symlink, `-f` replaces an existing one, and `-n` treats a link to a directory as a plain file. Without `-n`, `ln` would create a new link *inside* the old release directory instead of replacing `current`. After the swap, restart the service so it re-reads its files.

To prevent a repeat, make the deploy script verify the link with `readlink -f` and fail if it does not match the new release.

Common mistakes

- Editing a config in `/etc` without a backup, then forgetting what the original said
- Using `rm -rf` with an unquoted or empty variable
- Searching with `find /` on a busy server instead of a narrower directory
- Forgetting `-n` when replacing a symlink that points to a directory

Check yourself

Where do you normally find system-wide configuration files on Linux?

Answer: /etc

You run grep -rn "listen 443" /etc/nginx. What does it do?

Answer: Searches inside every file under /etc/nginx and prints matching lines with line numbers

A symbolic link's target directory is deleted. What happens to the link?

Answer: The link remains but is broken because its path points at nothing

Why use ln -sfn instead of ln -sf when replacing a link that points to a directory?

Answer: It stops ln from creating a new link inside the existing target directory

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 2. Users, groups and permissions

15 minutes. Read and set Linux permissions correctly, and debug the "permission denied" errors that trip up deployments.

A developer uploads a logo through your app. The file is on disk, the app can read it, but the web server returns `403 Forbidden`. Nothing is wrong with the code. The cause is almost always an ownership or permission problem, and you can diagnose it in two commands once you can read the permission bits.

What you will learn

- How users, groups and the `root` account relate
- How to read `rwx` and numeric modes such as `640`
- Why a directory needs the execute bit
- How to use `sudo` with least privilege

Users and groups

Every process runs as a user, and every file has an owner user and an owner group. Account data is stored in plain files: `/etc/passwd` (users), `/etc/group` (groups) and `/etc/shadow` (password hashes, readable only by root).

```
$ id
uid=1001(deploy) gid=1001(deploy) groups=1001(deploy),998(docker)
$ getent passwd www-data
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
```

`id` shows who you are and which groups you belong to. A shell of `/usr/sbin/nologin` marks a **service account**: it can own files and run a daemon, but nobody can log in as it. Use dedicated service accounts for each application so a compromise stays contained.

Reading permission bits

```
$ ls -l /srv/app/uploads
-rw-r--r-- 1 deploy deploy 48213 Oct 24 01:50 logo.png
```

The first column is ten characters: one for the type, then three triplets for **owner**, **group** and **others**. Each triplet is read (`r`), write (`w`) and execute (`x`).

Mode	Meaning	Typical use
`644` (`rw-r--r--`)	Owner reads and writes, everyone else reads	Config and web content
`600` (`rw-------`)	Owner only	SSH private keys, secrets
`755` (`rwxr-xr-x`)	Everyone can run or enter, only owner writes	Programs and directories
`750` (`rwxr-x---`)	Owner and group only	Shared app directories

Numbers add up per triplet: read is 4, write is 2, execute is 1. So `7` is `rwx`, `6` is `rw-`, `5` is `r-x`, and `4` is `r--`.

```
chmod 640 app.env              # owner rw, group r, others none
chmod u+x deploy.sh            # add execute for the owner
chown deploy:deploy app.env    # change owner and group
chmod -R g+rX /srv/app/static  # capital X adds execute only on directories
```

> **Note:** *On a* **directory**, `r` *lets you list names,* `w` *lets you create or delete entries, and* `x` *lets you enter it and reach things inside. Without* `x` *on every directory in a path, files below it are unreachable even if the file itself is world-readable.*

Least privilege with sudo

Do not work as root. Log in as yourself and elevate one command at a time.

```
$ sudo systemctl restart shop
```

MARGA by EknathaLabs marga.eknathalabs.com

```
$ sudo -l
User deploy may run the following commands on app01:
    (root) NOPASSWD: /usr/bin/systemctl restart shop
```

`sudo -l` lists what you may run. Granting a specific command, as above, is safer than a blanket `ALL`. Edit rules with `visudo`, which checks syntax before saving so you cannot lock yourself out with a typo.

Incident: uploads return 403 from nginx

> **Incident:** *nginx runs as `www-data` and serves files from `/srv/app/uploads`. New uploads, written by the app as user `deploy`, return 403 although the files exist and look readable.*

`namei -l` prints the permissions of every directory on the way to the file, which is the fastest way to find the broken link in the chain.

```
$ namei -l /srv/app/uploads/logo.png
f: /srv/app/uploads/logo.png
dr-xr-xr-x root   root   /
drwxr-xr-x root   root   srv
drwx------ deploy deploy app
drwxr-xr-x deploy deploy uploads
-rw-r--r-- deploy deploy logo.png
```

The directory `app` is `drwx------`: only `deploy` can enter it. `www-data` is "others" and has no `x`, so it never reaches `uploads`. The file's own permissions are fine. Grant traverse access on that one directory, then test as the web user:

```
$ sudo chmod o+x /srv/app
$ sudo -u www-data cat /srv/app/uploads/logo.png > /dev/null && echo readable
readable
```

`o+x` lets others enter `app` but not list its contents, which keeps the exposure small. Testing as the affected user proves the fix instead of assuming it.

> **Warning:** *Do not "fix" permission errors with `chmod -R 777`. It gives every user on the machine write access, hides the real cause and is a common source of security findings.*

Predict the permission

Diagram: How Linux decides if you may open a file

Part	What it does	What goes wrong
Process	A running program carries a user id and group ids.	If it runs as the wrong user, every check below uses the wrong identity.
Directory path	Each directory on the path needs execute (x) for the caller.	A missing x on one parent directory blocks access even if the file itself is open.
Owner, group or other	The kernel picks one class: owner first, then group, then other. Only that class is used.	Being the owner with fewer rights than the group still gives the owner the smaller set.
Permission bits	The r, w and x bits of the chosen class are compared with what is asked.	Wrong bits give Permission denied.
Allow or deny	The call succeeds or returns an error. ACLs and SELinux can add more checks.	A deny from SELinux looks like a permission error but chmod does not fix it.

Flow: Process to Directory path / Process to Owner, group or other / Directory path to Permission bits / Owner, group or other to Permission bits / Permission bits to Allow or deny

Common mistakes

- Forgetting that every parent directory needs the execute bit
- Running applications as root because it "just works"
- Leaving SSH private keys at `644`; `ssh` refuses keys that other users can read
- Using `777` instead of finding which user or group needs access

MARGA by EknathaLabs marga.eknathalabs.com

Check yourself

What does chmod 640 file set?

Answer: Owner read and write, group read, others no access

A file is mode 644 but the web server still cannot read it. What is the most likely cause?

Answer: A parent directory lacks the execute bit for the web server's user

Which mode is correct for an SSH private key?

Answer: 600

Why use visudo rather than editing /etc/sudoers with a normal editor?

Answer: It validates syntax before saving, so a typo cannot lock you out of sudo

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 3. Processes and services with systemd

15 minutes. Inspect and control processes, send the right signals, and manage services with systemd units and journalctl.

After a deploy, the new service will not stay up. `systemctl status` says "activating (auto-restart)" over and over. You cannot fix a crash loop by restarting it harder. You need to read why it exits. This lesson covers the process basics and the systemd workflow you will use daily.

What you will learn

- How to list and inspect processes
- What signals are and why `kill -9` is a last resort
- How a systemd unit file describes a service
- How to read a failing service's logs and exit codes

Processes in practice

A process is a running program with a process ID (PID), an owner and a parent. The kernel starts one process, PID 1, and every other process descends from it. On most modern distributions PID 1 is **systemd**.

```
$ ps -eo pid,ppid,user,%cpu,%mem,etime,cmd --sort=-%mem | head -4
    PID    PPID USER     %CPU %MEM     ELAPSED CMD
   2211     1 deploy    3.1 18.4  3-02:11:09 /opt/shop/current/bin/server --port 8080
    845     1 postgres  0.4  6.2 12-01:02:44 /usr/lib/postgresql/16/bin/postgres -D /var/lib/postg \
    resql/16/main
   1032     1 root      0.0  0.9 12-01:03:10 /usr/sbin/nginx -g daemon on; master_process on;
```

This is sample output. `ps -eo` lets you choose columns, and `--sort=-%mem` puts the biggest memory users first. `pgrep -a shop` finds processes by name, and `top` or `htop` shows a live view.

Signals

Signals are messages sent to a process. The ones you will use:

Signal	Number	Meaning
SIGTERM	15	Ask the process to shut down cleanly (the default for `kill`)
SIGINT	2	Same as pressing Ctrl+C
SIGHUP	1	Many daemons reload configuration on this
SIGKILL	9	Force-stop immediately; the process cannot catch it or clean up

```
kill 2211          # SIGTERM: lets the app finish requests and close files
kill -HUP 1032     # nginx reloads its configuration
kill -9 2211       # last resort: no cleanup, can leave locks and half-written data
```

Try `SIGTERM` first and wait. Reach for `SIGKILL` only when a process ignores `SIGTERM`.

Services with systemd

A **unit file** tells systemd how to run a service. Custom units go in `/etc/systemd/system/`.

```
[Unit]
Description=Shop API
After=network-online.target
Wants=network-online.target

[Service]
User=deploy
WorkingDirectory=/opt/shop/current
ExecStart=/opt/shop/current/bin/server --port 8080
Restart=on-failure
```

MARGA by EknathaLabs marga.eknathalabs.com

```
RestartSec=5

[Install]
WantedBy=multi-user.target
```

`ExecStart` must be an absolute path. `Restart=on-failure` restarts after a crash but not after a normal stop. `WantedBy` decides when the service starts at boot.

```
sudo systemctl daemon-reload      # required after editing a unit file
sudo systemctl enable --now shop  # start now and at every boot
sudo systemctl status shop        # state, recent log lines, exit code
sudo systemctl restart shop
journalctl -u shop -f             # follow the service's logs
```

Tip: `systemctl status` *shows the last few log lines, but* `journalctl -u shop -n 50 --no-pager` *shows more context. Combine* `-p err` *to show only errors.*

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Processes and services with systemd fits together

Part	What it does	What goes wrong
Processes in practice	A process is a running program with a process ID (PID), an owner and a parent.	Editing a unit file and forgetting systemctl daemon-reload.
Signals	Signals are messages sent to a process.	Using kill -9 as the first step.
Services with systemd	A unit file tells systemd how to run a service.	Reading only systemctl status and missing earlier errors in journalctl.

Flow: Processes in practice to Signals / Signals to Services with systemd

Incident: the service restarts every five seconds

Incident: *After a release,* `shop.service` *never becomes active. Users get connection refused.*

```
$ systemctl status shop
● shop.service - Shop API
     Loaded: loaded (/etc/systemd/system/shop.service; enabled)
     Active: activating (auto-restart) (Result: exit-code) since Sat 2026-10-24 02:31:18 UTC; 3s a \
    go
    Process: 4410 ExecStart=/opt/shop/current/bin/server --port 8080 (code=exited, status=203/EXEC \
    )
$ journalctl -u shop -n 3 --no-pager
Oct 24 02:31:18 app01 systemd[1]: shop.service: Failed at step EXEC spawning /opt/shop/current/bin \
    /server: No such file or directory
Oct 24 02:31:18 app01 systemd[1]: shop.service: Main process exited, code=exited, status=203/EXEC
Oct 24 02:31:18 app01 systemd[1]: shop.service: Failed with result 'exit-code'.
```

Exit status `203/EXEC` means systemd could not even start the binary. The message names the path, so check it:

```
$ ls -l /opt/shop/current/bin/
total 4
-rwxr-xr-x 1 deploy deploy 1873 Oct 24 02:29 start.sh
```

The new release ships `start.sh`, but the unit still points at `bin/server`. Update `ExecStart`, run `daemon-reload`, restart, and confirm:

```
$ sudo systemctl daemon-reload && sudo systemctl restart shop
$ systemctl is-active shop
active
```

The pattern is general: read the **exit code** and the **first error line**, not the restart loop. Common codes are `203/EXEC` (cannot execute the binary), `200/CHDIR` (bad `WorkingDirectory`), and `217/USER` (the `User=` does not exist).

MARGA by EknathaLabs marga.eknathalabs.com

Predict the signal

Common mistakes

- Editing a unit file and forgetting `systemctl daemon-reload`
- Using relative paths in `ExecStart`
- Using `kill -9` as the first step
- Reading only `systemctl status` and missing earlier errors in `journalctl`

Check yourself

What does kill 2211 send by default?

Answer: SIGTERM

Why is kill -9 a last resort?

Answer: The process cannot catch it, so it gets no chance to clean up and may leave locks or partial data

You edited /etc/systemd/system/shop.service. What must you run before restarting?

Answer: systemctl daemon-reload

A service exits with status=203/EXEC. What does that mean?

Answer: systemd could not execute the ExecStart binary, often a wrong path or missing execute permission

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 4. Disks and filesystems

15 minutes. Measure disk and inode usage, find what is eating space, and diagnose the "disk full but nothing to delete" mystery.

At 03:40 your database refuses writes with `No space left on device`. You run `df -h` and the root filesystem is 100% full. You add up the large directories and they total less than half the disk. Somebody has deleted a file that is still using space. This lesson gives you the tools to find it.

What you will learn

- How block devices, partitions, filesystems and mount points relate
- How to measure space and inodes
- How to find the largest directories quickly
- How a deleted-but-open file keeps consuming space

From device to directory

A disk appears as a **block device**, such as `/dev/sda` or `/dev/nvme0n1`. You split it into partitions, put a **filesystem** (ext4 or XFS are common) on each, and **mount** it at a directory.

```
$ lsblk
NAME   MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda      8:0    0   50G  0 disk
├─sda1   8:1    0   49G  0 part /
└─sda15  8:15   0  1.0G  0 part /boot/efi
sdb      8:16   0  200G  0 disk
└─sdb1   8:17   0  200G  0 part /var/lib/postgresql
```

This is sample output. `lsblk` shows the device tree, and `findmnt` shows what is mounted where. Permanent mounts are listed in `/etc/fstab`.

```
# /etc/fstab: device, mount point, type, options, dump, fsck order
UUID=3f5a1c0e-9c1e-4a77-8e0a-0c5d7a1b2c11  /var/lib/postgresql  ext4  defaults,noatime  0  2
```

> **Warning:** *A typo in `/etc/fstab` can stop a server from booting. Test new lines with `sudo mount -a` before you reboot, and consider the `nofail` option for non-critical mounts.*

Measuring space

```
$ df -h /
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        49G   49G     0 100% /
$ df -i /
Filesystem      Inodes   IUsed   IFree IUse% Mounted on
/dev/sda1      3276800  412887 2863913   13% /
```

`df -h` reports blocks used. `df -i` reports **inodes**, the records that describe each file. A filesystem can run out of inodes, for example from millions of tiny files, while having free blocks. Either one produces `No space left on device`.

To find what is large, limit `du` to one filesystem and sort:

```
$ sudo du -xh --max-depth=1 /var | sort -h | tail -4
312M    /var/cache
1.2G    /var/lib
3.4G    /var/log
5.1G    /var
```

`-x` stays on one filesystem, and `sort -h` understands sizes like `1.2G`. Drill down level by level with the same command.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Disks and filesystems fits together

Part	What it does	What goes wrong
From device to directory	A disk appears as a block device, such as /dev/sda or /dev/nvme0n1.	Running du on / without -x, which crosses into other mounts and /proc.
Measuring space	df -h reports blocks used. df -i reports inodes, the records that describe each file.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Creating and growing storage	For a new disk, the sequence is: partition, create a filesystem, mount, persist in fstab.	Editing /etc/fstab without testing mount -a.

Flow: From device to directory to Measuring space / Measuring space to Creating and growing storage

Incident: df says full, du disagrees

Incident: `df` *reports 49 GB used on* `/`*, but the largest directories add up to about 12 GB.*

Compare the two numbers. `df` asks the filesystem how many blocks are allocated. `du` adds up files it can see. The gap is space held by files that were **deleted while a process still has them open**. The name is gone, but the data stays until the last process closes it. A common cause is deleting a log file instead of rotating it.

```
$ sudo lsof +L1
COMMAND  PID   USER FD TYPE DEVICE  SIZE/OFF NLINK   NODE NAME
java    1432    app  5w  REG  253,0 37580963840     0 131075 /var/log/app/app.log (deleted)
```

`+L1` lists open files with fewer than one link, meaning deleted. Here process 1432 still holds a 35 GB log. Two ways to recover:

```
$ sudo systemctl restart app            # preferred: releases the file cleanly
$ sudo sh -c ': > /proc/1432/fd/5'      # truncate in place if you cannot restart yet
$ df -h /
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        49G   13G   34G  28% /
```

The fix is half the job. Prevent a repeat by rotating logs with `logrotate` (using `copytruncate` or a reload signal) so nobody has to delete live logs by hand, and alert at 80% disk usage so you act before 100%.

Creating and growing storage

For a new disk, the sequence is: partition, create a filesystem, mount, persist in `fstab`. Always check the device name twice, because formatting the wrong disk destroys data.

```
sudo parted /dev/sdc --script mklabel gpt mkpart primary ext4 0% 100%
sudo mkfs.ext4 /dev/sdc1
sudo mkdir -p /data && sudo mount /dev/sdc1 /data
sudo blkid /dev/sdc1          # copy the UUID into /etc/fstab
```

Many servers use **LVM** so volumes can grow without repartitioning. Growing is a two-step process: extend the logical volume, then grow the filesystem (for ext4, `resize2fs`).

Common mistakes

- Looking only at `df -h` and forgetting `df -i`
- Deleting a live log instead of truncating or rotating it
- Running `du` on `/` without `-x`, which crosses into other mounts and `/proc`
- Editing `/etc/fstab` without testing `mount -a`

Check yourself

df -h shows plenty of free space but writes fail with "No space left on device". What should you check next?

Answer: Inode usage with df -i

What does lsof +L1 list?

MARGA by EknathaLabs
marga.eknathalabs.com

Answer: Open files that have been deleted, which still hold disk space

Why use du -x when scanning /var?

Answer: It stays on one filesystem and does not cross into other mounts

What should you do before rebooting after editing /etc/fstab?

Answer: Run sudo mount -a to confirm every entry mounts without error

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 5. LVM and software RAID

16 minutes. Grow storage without downtime using LVM, protect against disk failure with mdadm RAID, and know which problem each one solves.

Partitions are rigid. When /var fills up, a plain partition forces you to back up, repartition and restore. **LVM** (Logical Volume Manager) adds a flexible layer between disks and filesystems, and **software RAID** protects data against disk failure. Production Linux servers commonly use one or both.

What you will learn

- The LVM layers: physical volumes, volume groups and logical volumes
- Creating, extending and snapshotting a logical volume
- RAID levels and when each is appropriate
- Replacing a failed disk in an mdadm array

The three LVM layers

Think of LVM as a pool of storage you carve up. A **physical volume (PV)** is a disk or partition handed to LVM. A **volume group (VG)** pools one or more PVs together. A **logical volume (LV)** is a slice of the pool that you format and mount like a partition, but that can later grow or shrink.

```
$ sudo pvcreate /dev/sdb /dev/sdc
$ sudo vgcreate data_vg /dev/sdb /dev/sdc
$ sudo lvcreate -n logs_lv -L 50G data_vg
$ sudo mkfs.ext4 /dev/data_vg/logs_lv
$ sudo mkdir -p /var/applogs
$ sudo mount /dev/data_vg/logs_lv /var/applogs
$ sudo pvs
$ sudo vgs
$ sudo lvs
```

Add a line to `/etc/fstab` using the device mapper path or the UUID from `blkid` so the volume mounts at boot. Use the `nofail` option for non-critical volumes so a missing disk does not stop the boot.

Growing a volume online

This is why LVM exists. If the volume group has free space, extend the logical volume and the filesystem in one command:

```
$ sudo vgs data_vg
$ sudo lvextend -r -L +20G /dev/data_vg/logs_lv
$ df -h /var/applogs
```

The `-r` flag resizes the filesystem too. If the group is full, add a new disk first with `pvcreate` and `vgextend data_vg /dev/sdd`. Growing is safe on a mounted ext4 or XFS filesystem. **Shrinking** is riskier: XFS cannot shrink at all, and ext4 must be unmounted and checked first.

Snapshots

An LVM snapshot freezes a point-in-time view of a volume using copy-on-write, handy before a risky upgrade or for a consistent backup.

```
$ sudo lvcreate -s -n logs_snap -L 5G /dev/data_vg/logs_lv
$ sudo mount -o ro /dev/data_vg/logs_snap /mnt/snap
$ sudo umount /mnt/snap && sudo lvremove -y /dev/data_vg/logs_snap
```

Snapshots are not backups. They live on the same disks, and a snapshot that fills its allocated space becomes invalid. Remove them when you are done.

MARGA by EknathaLabs marga.eknathalabs.com

RAID in one table

Level	Disks	Survives	Notes
RAID 0	2+	Nothing	Speed only, any disk failure loses everything
RAID 1	2	1 disk	Mirror, simple and safe
RAID 5	3+	1 disk	Capacity of n-1, slow rebuilds on big disks
RAID 6	4+	2 disks	Safer for large disks
RAID 10	4+	1 per mirror	Fast and resilient, half the capacity

RAID is about **availability**, not backup. It does not protect against deletion, corruption or ransomware.

```
$ sudo mdadm --create /dev/md0 --level=1 --raid-devices=2 /dev/sdb /dev/sdc
$ cat /proc/mdstat
$ sudo mdadm --detail /dev/md0
$ sudo mdadm --detail --scan | sudo tee -a /etc/mdadm/mdadm.conf
```

Replacing a failed disk

```
$ sudo mdadm --manage /dev/md0 --fail /dev/sdc
$ sudo mdadm --manage /dev/md0 --remove /dev/sdc
$ sudo mdadm --manage /dev/md0 --add /dev/sdd
$ watch cat /proc/mdstat
```

The array resynchronises in the background. Until it finishes you have no redundancy, so watch the progress and avoid heavy jobs.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How LVM and software RAID fits together

Part	What it does	What goes wrong
The three LVM layers	Think of LVM as a pool of storage you carve up.	Using up all free extents so you cannot extend or snapshot later.
Growing a volume online	This is why LVM exists.	Trying to shrink XFS.
Snapshots	An LVM snapshot freezes a point-in-time view of a volume using copy-on-write, handy before a risky upgrade or for a consistent backup.	Letting a snapshot fill up and become invalid.
RAID in one table	RAID is about availability, not backup. It does not protect against deletion, corruption or ransomware.	Treating RAID as a backup.
Replacing a failed disk	The array resynchronises in the background.	Forgetting to update fstab or mdadm.conf, so volumes vanish after a reboot.

Flow: The three LVM layers to Growing a volume online / Growing a volume online to Snapshots / Snapshots to RAID in one table / RAID in one table to Replacing a failed disk

Incident: /var is full and the disk is not

> **Incident:** *The database host stops writing because /var is 100% full. The server has a free 200 GB disk, but /var is a plain partition on a different disk, and the team plans a maintenance window to repartition.*

If /var had been an LVM volume, the fix would take a minute. The team can still reduce risk: stop the writer, move the data directory to a new LVM volume built on the free disk, and mount it in place. Going forward, put growth-prone paths such as /var, /var/lib/docker and /data on logical volumes and leave 10 to 20 percent of each volume group unallocated for emergencies.

MARGA by EknathaLabs
marga.eknathalabs.com

```
$ sudo vgs
  VG      #PV #LV #SN Attr   VSize   VFree
  data_vg   2   2   0 wz--n- 400.00g 120.00g
$ sudo lvextend -r -L +50G /dev/data_vg/db_lv
```

Common mistakes

- Treating RAID as a backup
- Using up all free extents so you cannot extend or snapshot later
- Forgetting to update fstab or mdadm.conf, so volumes vanish after a reboot
- Letting a snapshot fill up and become invalid
- Trying to shrink XFS

Check yourself

What is a logical volume?

Answer: A slice of a volume group that is formatted and mounted like a partition but can be resized

Which command grows a logical volume and its filesystem together?

Answer: lvextend -r -L +20G

Which statement about RAID is true?

Answer: RAID improves availability but does not protect against deletion or corruption

Why keep some free space in a volume group?

Answer: You can extend volumes or take snapshots during an emergency

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 6. Package management and repositories

14 minutes. Install, update, pin and verify software with apt and dnf, understand repositories and signatures, and keep servers patched without surprises.

Every package on a server is code that runs with privilege. Package managers install software, resolve dependencies, verify signatures and make upgrades repeatable. Knowing them well is the difference between controlled patching and hoping nothing breaks.

What you will learn

- The apt and dnf workflows for install, search, update and remove
- How repositories, GPG keys and signatures protect you
- Holding a package at a version
- Patching safely and checking what changed

Two families

Debian and Ubuntu use **apt** with `.deb` packages. RHEL, Rocky, Alma, Fedora and Amazon Linux use **dnf** (older systems use `yum`) with `.rpm` packages. The concepts are the same.

```
$ sudo apt update
$ apt search nginx | head -5
$ apt show nginx | head -12
$ sudo apt install -y nginx
$ dpkg -L nginx | head
$ dpkg -S /usr/sbin/nginx
$ sudo apt remove nginx
$ sudo apt purge nginx
```

`apt update` refreshes the package index, it does not upgrade anything. `remove` keeps configuration files, `purge` deletes them too. `dpkg -S` tells you which package owns a file.

```
$ sudo dnf makecache
$ dnf search nginx
$ dnf info nginx
$ sudo dnf install -y nginx
$ rpm -ql nginx | head
$ rpm -qf /usr/sbin/nginx
$ dnf history
```

Repositories and trust

A repository is a server of packages plus an index. Your system trusts a repository only if you installed its **signing key**, and every package or index is verified against that key. Never disable signature checks to "make an install work".

```
$ ls /etc/apt/sources.list.d/
$ cat /etc/yum.repos.d/*.repo | head -15
$ sudo apt-key list 2>/dev/null | head -3
$ apt-cache policy nginx
```

Add third-party repositories deliberately: download the key over HTTPS, store it in `/etc/apt/keyrings/`, and reference it with `signed-by=` in the source entry. Every extra repository widens who can put code on your servers, so keep the list short and review it.

Pinning and holding

Production rarely wants the newest version of everything. Hold a package so routine upgrades skip it, or install an exact version.

```
$ apt-cache madison nginx | head -3
$ sudo apt install -y nginx=1.24.0-2ubuntu7
$ sudo apt-mark hold nginx
$ apt-mark showhold
```

MARGA by EknathaLabs marga.eknathalabs.com

```
$ sudo dnf install -y nginx-1.24.0
$ sudo dnf versionlock add nginx
```

A hold must be temporary and tracked. A package held for two years is a security risk with a ticket nobody reads.

Patching safely

```
$ apt list --upgradable
$ sudo apt upgrade -y
$ sudo apt autoremove -y
$ ls /var/run/reboot-required
$ dnf check-update
$ sudo dnf upgrade --security -y
$ sudo needs-restarting -r
```

A reasonable routine is: patch a staging host, run smoke tests, roll through production in small batches behind a load balancer, and reboot where the kernel or libc changed. Enable **unattended security updates** for low-risk hosts, with alerts when they fail.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Package management and repositories fits together

Part	What it does	What goes wrong
Two families	Debian and Ubuntu use apt with .deb packages.	Running apt upgrade on production without testing first.
Repositories and trust	A repository is a server of packages plus an index.	Disabling GPG checks or using --allow-unauthenticated.
Pinning and holding	Production rarely wants the newest version of everything.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Patching safely	A reasonable routine is: patch a staging host, run smoke tests, roll through production in small batches behind a load balancer, and reboot where the kernel or libc...	Forgetting to restart services or reboot after library and kernel updates.

Flow: Two families to Repositories and trust / Repositories and trust to Pinning and holding / Pinning and holding to Patching safely

Incident: the held package and the CVE

> **Incident:** *A security scanner flags an OpenSSL vulnerability on forty servers. Patch automation reports success, but the scanner still fails them. Investigation shows* `apt-mark hold openssl` *was set a year ago during a compatibility problem and never removed.*

Find holds across the fleet and decide on each one:

```
$ apt-mark showhold
openssl
$ sudo apt-mark unhold openssl
$ sudo apt install --only-upgrade openssl
$ openssl version
```

Then fix the real issue, which was an application depending on an old library, and add a quarterly review that lists every hold with an owner and a reason.

Common mistakes

- Running `apt upgrade` on production without testing first
- Disabling GPG checks or using `--allow-unauthenticated`
- Adding random repositories copied from a blog post

MARGA by EknathaLabs marga.eknathalabs.com

- Leaving holds in place forever
- Forgetting to restart services or reboot after library and kernel updates

Check yourself

What does apt update do?

Answer: Refreshes the list of available packages from the repositories

Why are repository signing keys important?

Answer: They let the system verify packages really come from the repository owner

How do you find which package owns /usr/sbin/nginx on Debian?

Answer: dpkg -S /usr/sbin/nginx

What is the danger of leaving apt-mark hold in place?

Answer: The package misses security fixes and the hold is forgotten

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 7. SSH and secure remote access

15 minutes. Use key-based SSH, configure clients and servers safely, tunnel traffic, jump through bastion hosts and transfer files with scp and rsync.

SSH is how you reach nearly every Linux server. It encrypts the session, authenticates with keys and can carry file transfers and network tunnels. A weak SSH setup is one of the most common ways servers get compromised.

What you will learn

- Generating and using key pairs
- Writing `~/.ssh/config` so connections are short and consistent
- Hardening sshd and using bastion (jump) hosts
- Port forwarding, scp and rsync

Keys instead of passwords

A key pair has a private key that never leaves your machine and a public key placed on servers. Use the modern Ed25519 type and protect the private key with a passphrase.

```
$ ssh-keygen -t ed25519 -C "reddy@laptop" -f ~/.ssh/id_ed25519
$ ssh-copy-id -i ~/.ssh/id_ed25519.pub deploy@10.0.1.20
$ ssh deploy@10.0.1.20
$ ssh-add ~/.ssh/id_ed25519
$ ssh-add -l
```

`ssh-agent` holds the unlocked key in memory so you type the passphrase once. On first connection SSH records the server's fingerprint in `~/.ssh/known_hosts`. If the fingerprint later changes, SSH warns loudly. Do not blindly delete the line: first confirm whether the server was rebuilt.

The client config

```
Host bastion
    HostName 203.0.113.10
    User ops
    IdentityFile ~/.ssh/id_ed25519

Host app-*
    User deploy
    ProxyJump bastion
    IdentityFile ~/.ssh/id_ed25519
    ServerAliveInterval 30

Host *
    AddKeysToAgent yes
    IdentitiesOnly yes
```

With this, `ssh app-1` connects through the bastion automatically. **ProxyJump** is the safe way to reach private servers: your private key stays on your laptop and never sits on the bastion, unlike agent forwarding to untrusted hosts.

Hardening the server

Edit `/etc/ssh/sshd_config` (or a file in `sshd_config.d/`) and test before reloading.

```
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
AllowUsers deploy ops
MaxAuthTries 3
ClientAliveInterval 300

$ sudo sshd -t
$ sudo systemctl reload sshd
$ sudo ss -tlnp | grep sshd
$ sudo journalctl -u ssh --since "1 hour ago" | grep -i failed | tail
```

MARGA by EknathaLabs marga.eknathalabs.com

Always keep an existing session open while you test a new one, so a typo does not lock you out. Add rate limiting with `fail2ban` or a cloud security group that limits port 22 to known addresses.

Tunnels and file transfer

```
$ ssh -L 5433:db.internal:5432 app-1
$ ssh -N -D 1080 bastion
$ scp ./release.tar.gz app-1:/tmp/
$ rsync -avz --delete ./site/ app-1:/var/www/site/
$ rsync -avzn ./site/ app-1:/var/www/site/
```

`-L` forwards a local port to a remote destination, useful for reaching a private database from your laptop. `rsync` copies only differences, and `-n` is a dry run, which you should use before any `--delete`.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How SSH and secure remote access fits together

Part	What it does	What goes wrong
Keys instead of passwords	A key pair has a private key that never leaves your machine and a public key placed on servers.	Permissions on ~/.ssh or the private key that are too open (SSH refuses them).
The client config	With this, ssh app-1 connects through the bastion automatically.	Forwarding the agent to hosts you do not fully trust.
Hardening the server	Edit /etc/ssh/sshd_config (or a file in sshd_config.d/) and test before reloading.	Editing sshd_config and reloading without testing.
Tunnels and file transfer	-L forwards a local port to a remote destination, useful for reaching a private database from your laptop.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Keys instead of passwords to The client config / The client config to Hardening the server / Hardening the server to Tunnels and file transfer

MARGA by EknathaLabs marga.eknathalabs.com

Incident: the leaked key on a shared bastion

> **Incident:** *Engineers copy their private keys to a shared bastion so they can hop to internal servers. A contractor's account on the bastion is compromised, and the attacker reads every key in every home directory.*

Private keys were stored where other people and processes could reach them. Remediation and prevention:

```
$ sudo find /home -name "id_*" -not -name "*.pub" 2>/dev/null
$ sudo chmod 700 /home/*/.ssh
$ sudo sed -i 's/^#\?PasswordAuthentication.*/PasswordAuthentication no/' /etc/ssh/sshd_config
```

Rotate every exposed key, remove old entries from `authorized_keys`, switch to `ProxyJump` so keys stay on laptops, and consider short-lived certificates or a cloud session manager so there are no long-lived keys at all.

Common mistakes

- Using password authentication on internet-facing hosts
- Permissions on `~/.ssh` or the private key that are too open (SSH refuses them)
- Forwarding the agent to hosts you do not fully trust
- Editing sshd_config and reloading without testing
- Deleting a `known_hosts` warning without investigating

Check yourself

Which key type is a good modern default?

Answer: Ed25519

What is the safest way to reach private servers through a bastion?

Answer: Use ProxyJump so the key stays on your machine

Why run sshd -t before reloading?

Answer: It validates the configuration so a typo does not lock you out

What does ssh -L 5433:db.internal:5432 app-1 do?

Answer: Forwards local port 5433 to db.internal port 5432 via app-1

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 8. Scheduling with cron and systemd timers

14 minutes. Run jobs on a schedule with cron and systemd timers, avoid the classic environment traps, and make scheduled work observable.

Backups, certificate renewals, log cleanup and reports all run on a schedule. Scheduled jobs fail silently more often than any other kind of automation, so the way you set them up matters as much as what they do.

What you will learn

- Cron syntax and where crontabs live
- Why cron jobs fail with "works in my shell"
- systemd timers and when they are better
- Locking, logging and alerting for scheduled jobs

Cron syntax

A crontab line has five time fields and a command: minute, hour, day of month, month, day of week.

```
# m  h  dom mon dow  command
*/5  *  *   *   *    /usr/local/bin/check-queue.sh
30   2  *   *   1-5  /usr/local/bin/backup.sh
0    4  1   *   *    /usr/local/bin/monthly-report.sh
@reboot               /usr/local/bin/warm-cache.sh
```

```
$ crontab -l
$ crontab -e
$ sudo crontab -u deploy -l
$ ls /etc/cron.d /etc/cron.daily
$ sudo grep CRON /var/log/syslog | tail -5
```

Per-user crontabs are edited with `crontab -e`. System-wide jobs live in `/etc/cron.d/` and have an extra **user** field after the time fields. Use a site like crontab.guru to double-check an unfamiliar expression, and remember that cron uses the server's timezone, which in the cloud is usually UTC.

Why cron jobs fail

Cron starts jobs with a **tiny environment**: a minimal PATH, no profile files, no interactive variables, and the current directory is the home directory. A script that works when you run it by hand can fail under cron.

```
SHELL=/bin/bash
PATH=/usr/local/bin:/usr/bin:/bin
MAILTO=ops@example.com

30 2 * * * /opt/backup/run.sh >> /var/log/backup.log 2>&1
```

Set `PATH` explicitly, use absolute paths for every command and file, and redirect output. Output that goes nowhere is lost, and output that goes to local mail is usually never read.

Preventing overlap with locks

If a job takes longer than its interval, cron starts a second copy on top of the first. Use `flock` so only one runs at a time:

```
$ flock -n /var/lock/backup.lock /opt/backup/run.sh || echo "previous run still active"
```

In a crontab: `*/10 * * * * flock -n /var/lock/sync.lock /opt/sync.sh`. The `-n` flag makes the second instance exit immediately instead of waiting.

MARGA by EknathaLabs
marga.eknathalabs.com

systemd timers

A timer is a unit that triggers a service unit. You get journal logging, dependency handling, resource limits and a status command for free.

```
[Unit]
Description=Nightly backup

[Service]
Type=oneshot
ExecStart=/opt/backup/run.sh
User=backup

[Unit]
Description=Run backup nightly

[Timer]
OnCalendar=*-*-* 02:30:00
Persistent=true
RandomizedDelaySec=300

[Install]
WantedBy=timers.target
```

Save them as `backup.service` and `backup.timer` in `/etc/systemd/system/`.

```
$ sudo systemctl daemon-reload
$ sudo systemctl enable --now backup.timer
$ systemctl list-timers --all
$ systemctl status backup.service
$ journalctl -u backup.service --since today
$ systemd-analyze calendar "Mon..Fri 02:30"
```

`Persistent=true` runs a missed job after the machine comes back from downtime, which cron does not do. `RandomizedDelaySec` spreads load when many hosts share a schedule.

Know when it did not run

A job that never ran is invisible. Make success observable: have the job ping a monitoring URL or write a metric on success, and alert when the signal is missing for longer than expected (a dead man's switch).

```
$ /opt/backup/run.sh && curl -fsS -m 10 https://hc.example.com/ping/abc123
```

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Scheduling with cron and systemd timers fits together

Part	What it does	What goes wrong
Cron syntax	A crontab line has five time fields and a command: minute, hour, day of month, month, day of week.	Assuming the cron timezone matches yours.
Why cron jobs fail	Cron starts jobs with a tiny environment: a minimal PATH, no profile files, no interactive variables, and the current directory is the home directory.	Relying on the interactive PATH and environment.
Preventing overlap with locks	If a job takes longer than its interval, cron starts a second copy on top of the first.	Overlapping runs of a slow job.
systemd timers	A timer is a unit that triggers a service unit.	Having no alert when the job stops running.
Know when it did not run	A job that never ran is invisible. Make success observable: have the job ping a monitoring URL or write a metric on success, and alert when the signal is missing for...	Know when it did not run A job that never ran is invisible.

Flow: Cron syntax to Why cron jobs fail / Why cron jobs fail to Preventing overlap with locks / Preventing overlap with locks to systemd timers / systemd timers to Know when it did not run

MARGA by EknathaLabs marga.eknathalabs.com

Incident: the certificate that did not renew

> **Incident:** *The certificate renewal cron job has been failing for two months. It worked when tested by hand. Now the production certificate expires at midnight and customers see browser warnings.*

The job used `certbot` from `/usr/local/bin`, which cron's default PATH did not include, and output was mailed to a root mailbox nobody reads. Fix it properly:

```
$ sudo grep -r certbot /etc/cron.d /var/spool/cron 2>/dev/null
$ sudo certbot renew --dry-run
$ sudo systemctl list-timers | grep certbot
```

Use an absolute path or set PATH, log to a file the monitoring agent watches, run `renew --dry-run` in CI or weekly, and add an alert that fires when any certificate has fewer than 21 days left. Renewal should never depend on a human noticing an email.

Common mistakes

- Relying on the interactive PATH and environment
- Not redirecting output, so failures leave no trace
- Overlapping runs of a slow job
- Assuming the cron timezone matches yours
- Having no alert when the job stops running

Check yourself

Why does a script work in your shell but fail in cron?

Answer: Cron runs with a minimal environment, including a short PATH

What does flock -n do in a cron line?

Answer: Skips the run if a previous run still holds the lock

What does Persistent=true do in a systemd timer?

Answer: Runs a missed job when the system comes back up

How do you detect that a scheduled job silently stopped?

Answer: Alert when its success signal or heartbeat is missing

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 9. Logs and troubleshooting

15 minutes. Read journald and file logs, correlate events by time, and use the kernel log to explain mysterious restarts.

Users report random `502 Bad Gateway` errors around 14:03. The web server looks healthy and CPU is idle. The answer is in the logs, but only if you know which logs to read, how to narrow them to a time window, and how to connect two sources. This lesson builds that routine.

What you will learn

- Where Linux keeps logs
- How to filter `journalctl` by unit, time and severity
- How to correlate application, web server and kernel logs
- How to spot an out-of-memory kill

Two places logs live

Modern systems use **journald**, queried with `journalctl`, for services run by systemd. Many programs also write plain files under `/var/log`.

Source	Look here	For
journald	`journalctl -u SERVICE`	Anything started by systemd
`/var/log/nginx/`	`access.log`, `error.log`	Web requests and proxy errors
`/var/log/auth.log` (Debian family) or `/var/log/secure` (Red Hat family)	Logins and sudo use	Security events
Kernel ring buffer	`dmesg -T`	Hardware, disk and out-of-memory events

journalctl filters you will use constantly

```
journalctl -u shop --since "14:00" --until "14:10"   # one service, one window
journalctl -u shop -p err -n 50 --no-pager           # errors and worse, last 50
journalctl -b -1 -p err                              # errors from the previous boot
journalctl -f                                        # follow everything live
```

Severity runs from `emerg` (0) to `debug` (7). `-p err` shows `err` and anything more severe.

Searching plain log files

```
$ grep -c " 502 " /var/log/nginx/access.log
41
$ awk '$9 == 502 {print $4}' /var/log/nginx/access.log | cut -c2-15 | sort | uniq -c
     41 24/Oct/2026:14
$ tail -n 3 /var/log/nginx/error.log
2026/10/24 14:03:11 [error] 1032#1032: *9021 connect() failed (111: Connection refused) while conn \
    ecting to upstream, upstream: "http://127.0.0.1:8080/"
```

The default nginx access log puts the status code in field 9, so `awk '$9 == 502'` filters on it. Always include the time field when you aggregate, because the pattern in time is the clue. `less +F file` follows a file like `tail -f` but lets you scroll back.

> **Tip:** *Make timestamps comparable before correlating. Check the time zone with* `timedatectl`*. Mixing UTC and local time is a classic way to look at the wrong minute.*

Incident: 502 errors every few hours

Incident: `502` *responses cluster at 14:03. nginx reports* `Connection refused` *to the upstream on port 8080.*

"Connection refused" means nothing was listening on 8080 at that moment. Why not? Ask the service's journal for the same minute:

```
$ journalctl -u shop --since "14:00" --until "14:10" --no-pager
Oct 24 14:03:05 app01 systemd[1]: shop.service: Main process exited, code=killed, status=9/KILL
```

MARGA by EknathaLabs marga.eknathalabs.com

```
Oct 24 14:03:05 app01 systemd[1]: shop.service: Failed with result 'signal'.
Oct 24 14:03:10 app01 systemd[1]: shop.service: Scheduled restart job, restart counter is at 1.
Oct 24 14:03:10 app01 systemd[1]: Started Shop API.
```

The process did not crash by itself. It was killed with signal 9 and systemd restarted it five seconds later, which matches the five-second window of refused connections. Nobody ran `kill -9`, so ask the kernel:

```
$ sudo dmesg -T | grep -i -E "out of memory|killed process"
[Sat Oct 24 14:03:05 2026] Out of memory: Killed process 2211 (server) total-vm:6291456kB, anon-rs \
    s:3932160kB, file-rss:0kB, shmem-rss:0kB
```

The kernel's **OOM killer** chose the application because the machine ran out of memory. Confirm the trend with `free -h` and check whether the service has a memory limit. Real fixes are to reduce the application's memory use, add memory, or set a cgroup limit (`MemoryMax=` in the unit) so only this service is affected. Restarting more often hides the problem.

The method is the point: **symptom** (502), **proximate cause** (connection refused), **event** (process killed), **root cause** (OOM). Each step used a different log.

Log rotation

Logs grow without bound unless rotated. `logrotate` renames, compresses and deletes old logs on a schedule, configured in `/etc/logrotate.d/`.

```
/var/log/app/*.log {
    daily
    rotate 14
    compress
    delaycompress
    missingok
    notifempty
    copytruncate
}
```

`copytruncate` copies the log and empties the original so programs that keep the file open do not need a restart. Test a config with `logrotate -d /etc/logrotate.d/app`, which prints what would happen without doing it.

Diagram: Follow a failing service through the logs

Part	What it does	What goes wrong
Symptom	A user reports an error or a monitor fires.	Without a time and a name, you search blindly.
systemctl status	Shows state, exit code and the last log lines of the unit.	A recent restart can hide the first failure; look further back.
journalctl -u	Shows all unit logs with timestamps. Use --since to narrow.	If the journal is not persistent, logs from before a reboot are gone.
Cause	Compare the first error with the last change: a config, a package or a disk.	Fixing the later error instead of the first one leads you away.
Fix and verify	Change one thing, restart, and watch the logs confirm the fix.	Changing several things at once makes it unclear what worked.

Flow: Symptom to systemctl status / Symptom to journalctl -u / systemctl status to Cause / journalctl -u to Cause / Cause to Fix and verify

Common mistakes

- Reading only the latest lines instead of the minute when the problem occurred
- Comparing logs without confirming they use the same time zone
- Stopping at "the process died" without asking what killed it
- Deleting log files by hand instead of configuring rotation

MARGA by EknathaLabs marga.eknathalabs.com

Check yourself

Which command shows only errors for the shop service between 14:00 and 14:10?

Answer: journalctl -u shop -p err --since "14:00" --until "14:10"

nginx logs "connect() failed (111: Connection refused) while connecting to upstream". What does it indicate?

Answer: Nothing was listening on the upstream address and port

A service log shows "status=9/KILL" and nobody ran kill. Where do you look next?

Answer: The kernel log with dmesg -T for an out-of-memory kill

Why does the logrotate example use copytruncate?

Answer: It lets programs that keep the log file open continue writing without a restart

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 10. Performance triage

16 minutes. Use load average, vmstat and iostat to tell CPU, memory and disk problems apart in under five minutes.

The server "feels slow". Dashboards are not available yet, and you have a shell. A good engineer does not start guessing or restarting things. They run the same short checklist every time and let the numbers decide whether the bottleneck is CPU, memory, disk or network.

What you will learn

- How to read load average correctly
- A five-command triage sequence
- How to separate CPU saturation from disk waiting
- The USE method: utilization, saturation, errors

Load average is a queue length

```
$ uptime
 02:44:10 up 12 days,  3:02,  2 users,  load average: 11.86, 9.40, 6.12
$ nproc
4
```

The three numbers are the average count of tasks that were running or waiting to run over the last 1, 5 and 15 minutes. On Linux this includes tasks blocked on disk I/O. Compare them with the number of CPU cores from `nproc`. A load of 4 on a 4-core machine is full but healthy. A load of 11.86 on 4 cores means work is queueing, and the 1-minute number being higher than the 15-minute one means it is getting worse.

Load alone does not say **why**. You need the next commands.

The USE method

For each resource (CPU, memory, disk, network) ask three questions:

Question	Meaning	Example evidence
Utilization	How busy is it?	CPU 95% busy
Saturation	Is work waiting?	Run queue longer than cores, swapping
Errors	Is it failing?	Disk errors in `dmesg`, packet drops

Checking all three for each resource stops you from fixating on the first thing that looks odd.

A five-command triage

```
uptime              # 1. load trend
vmstat 1 5          # 2. CPU, memory, swap and I/O wait together
free -h             # 3. memory, focusing on "available"
iostat -xz 1 3      # 4. per-disk utilization and latency (package: sysstat)
ss -s               # 5. socket totals; then top for the process view
```

`free -h` has a column named `available`. It estimates how much memory can be used without swapping, including reclaimable cache. Low `free` with a large `available` is normal, because Linux uses spare memory as file cache.

Reading vmstat

```
$ vmstat 1 3
procs -----------memory---------- ---swap-- -----io---- -system-- ------cpu-----
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st
 1  7      0 213456  10240 812344    0    0  5120   980 1200 2400  4  3 12 81  0
 0  8      0 211320  10240 814012    0    0  4980  1010 1180 2350  3  2 14 81  0
```

This is sample output. Focus on a few columns:

- `r`: tasks waiting to run on the CPU. Persistently above the core count means CPU saturation.
- `b`: tasks blocked, usually waiting on disk.
- `si`/`so`: memory swapped in and out. Sustained non-zero values mean memory pressure.
- `us`/`sy`: CPU time in user code and kernel code. `id` is idle.
- `wa`: CPU time spent waiting for I/O to complete.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Performance triage fits together

Part	What it does	What goes wrong
Load average is a queue length	The three numbers are the average count of tasks that were running or waiting to run over the last 1, 5 and 15 minutes.	Reading load average without comparing it to the number of cores.
The USE method	For each resource (CPU, memory, disk, network) ask three questions.	Treating low free memory as a problem when available is large.
A five-command triage	free -h has a column named available. It estimates how much memory can be used without swapping, including reclaimable cache.	Looking at averages only, instead of watching vmstat 1 for a few seconds.
Reading vmstat	This is sample output.	Check the Common mistakes list below this diagram for what usually goes wrong here.
What to check when it is not the	CPU saturated: high us and r above core count.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Load average is a queue length to The USE method / The USE method to A five-command triage / A five-command triage to Reading vmstat / Reading vmstat to What to check when it is not the

Incident: load is 12 but the CPUs are idle

Incident: *The reporting server has load 11.86 on 4 cores, yet users say only exports are slow.*

In the `vmstat` output above, `r` is 1 and `id` is 12, so the CPU is not the limit. But `b` is 7 and `wa` is 81. Seven tasks are blocked and 81% of CPU time is spent waiting for I/O. This is a **disk** problem. Find which device with `iostat`:

```
$ iostat -xz 1 2
Device   r/s     w/s    rkB/s    wkB/s  r_await  w_await  aqu-sz  %util
sda     2.0     4.0     16.0     48.0     1.1      2.0    0.01    0.5
sdb   410.0    35.0 52480.0  4400.0    38.5     12.2   16.90   99.8
```

`sdb` is 99.8% utilized with a 38 ms read wait and a deep queue (`aqu-sz`). That is the database volume. Now find the process doing the reading:

```
$ sudo iotop -oP -n 1
  PID  PRIO  USER   DISK READ  DISK WRITE  COMMAND
 3187  be/4  app    48.20 M/s     0.00 B/s  python3 /opt/report/export.py
```

A nightly export is scanning large tables at the same time as users query. Options include scheduling the job off-peak, lowering its priority with `ionice -c3`, adding an index so it reads less, or moving it to a read replica. The triage did not require guessing, because each command removed a hypothesis.

Why it matters: *Rebooting would have cleared the symptom for an hour and taught you nothing. A five-minute triage gives you a cause you can fix and explain.*

What to check when it is not the disk

- **CPU saturated:** high `us` and `r` above core count. Use `top` sorted by CPU (`P`) and look for a runaway process.

MARGA by EknathaLabs marga.eknathalabs.com

- **Memory pressure:** `si`/`so` non-zero, `available` low, OOM events in `dmesg`.
- **Network:** `ss -s` for socket counts, `ip -s link` for errors and drops.

Common mistakes

- Reading load average without comparing it to the number of cores
- Treating low `free` memory as a problem when `available` is large
- Restarting the service before capturing evidence
- Looking at averages only, instead of watching `vmstat 1` for a few seconds

Check yourself

A 4-core server shows load average 11.86. What is the most accurate interpretation?

Answer: Roughly three times as many tasks want to run or are blocked as there are cores, so work is queueing

In vmstat, the wa column is 81. What does that suggest?

Answer: The CPU is spending most of its time waiting for disk I/O

free -h shows only 200 MB in the free column but 9 GB in available. What does that mean?

Answer: Plenty of memory is available because much of it is reclaimable cache

In the USE method, what does saturation mean?

Answer: Whether work is waiting because the resource is fully used

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 11. Boot process, kernel and recovery

15 minutes. Follow the boot sequence from firmware to systemd, inspect and tune the kernel with sysctl and modules, and recover a server that will not boot.

When a server does not come back after a reboot, you have no SSH session, no logs from the usual tools and a console that shows only a few lines. Knowing the boot sequence tells you where to look.

What you will learn

- The stages of the Linux boot process
- Reading boot logs and unit failures
- Kernel parameters, modules and `sysctl`
- Recovering with rescue mode and fixing fstab errors

The sequence

1. **Firmware** (UEFI or BIOS) runs hardware checks and finds a boot device.
2. The **bootloader** (GRUB) loads the kernel and an initial RAM filesystem (initramfs).
3. The **kernel** initialises hardware, mounts the root filesystem and starts PID 1.
4. **systemd** (PID 1) mounts filesystems from fstab, starts services in dependency order and reaches a target such as `multi-user.target`.

Each stage has its own failure signature. A GRUB prompt means the bootloader or its config is broken. A kernel panic mentioning "unable to mount root fs" means the initramfs or root device is wrong. A hang with "A start job is running for dev-disk-by-uuid" means systemd is waiting for a filesystem in fstab that does not exist.

Reading the boot

```
$ systemd-analyze
$ systemd-analyze blame | head
$ systemd-analyze critical-chain
$ journalctl -b -p err
$ journalctl -b -1 -p err | tail -20
$ systemctl --failed
$ dmesg --level=err,warn | tail
```

`journalctl -b -1` shows the previous boot, which is essential after a crash. If the journal is not persistent, create `/var/log/journal` and restart journald so logs survive reboots.

The kernel and its tunables

```
$ uname -r
$ lsmod | head -5
$ modinfo overlay | head -5
$ sudo modprobe br_netfilter
$ sysctl net.ipv4.ip_forward
$ sudo sysctl -w net.ipv4.ip_forward=1
$ cat /proc/cmdline
```

`sysctl -w` changes a value until reboot. To keep it, put it in a file under `/etc/sysctl.d/`:

```
net.ipv4.ip_forward = 1
vm.swappiness = 10
fs.inotify.max_user_watches = 524288
net.core.somaxconn = 4096

$ sudo sysctl --system
$ cat /etc/modules-load.d/k8s.conf
```

Kubernetes nodes, for example, need `br_netfilter` loaded and IP forwarding enabled. Modules listed in `/etc/modules-load.d/` load automatically at boot.

MARGA by EknathaLabs marga.eknathalabs.com

Recovery

If the system drops to an emergency shell, the root filesystem is usually mounted read-only. Check what failed, then fix it:

```
$ journalctl -xb | tail -30
$ mount -o remount,rw /
$ blkid
$ nano /etc/fstab
$ mount -a
$ systemctl default
```

If you cannot get a shell at all, boot with a rescue ISO, mount the root volume at `/mnt` and `chroot` into it. In the cloud, detach the root volume, attach it to a healthy instance, fix the file and reattach it. At the GRUB menu, pressing `e` lets you edit the kernel line once, for example adding `systemd.unit=rescue.target` to reach single-user mode.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Boot process, kernel and recovery fits together

Part	What it does	What goes wrong
The sequence	Firmware (UEFI or BIOS) runs hardware checks and finds a boot device.	Editing fstab without running mount -a to test it.
Reading the boot	journalctl -b -1 shows the previous boot, which is essential after a crash.	Leaving the journal non-persistent, so crash logs disappear.
The kernel and its tunables	sysctl -w changes a value until reboot.	Changing sysctl values with -w and forgetting to persist them.
Recovery	If the system drops to an emergency shell, the root filesystem is usually mounted read-only.	Updating the kernel without keeping a previous one to fall back to.

Flow: The sequence to Reading the boot / Reading the boot to The kernel and its tunables / The kernel and its tunables to Recovery

Incident: the reboot that never came back

> **Incident:** *After patching, a database server does not respond. The cloud console shows it running, but the serial console is stuck on "A start job is running for /dev/disk/by-uuid/..." for 1m 30s repeated. Someone had replaced a data disk last week, and its old UUID is still in fstab.*

systemd waits for a device that will never appear. Boot to emergency or rescue mode and correct the entry:

```
$ blkid | grep data
$ sudo nano /etc/fstab
$ sudo mount -a && echo fstab ok
```

Prevent a repeat by testing with `mount -a` after every fstab edit, using `nofail,x-systemd.device-timeout=10s` on non-critical mounts, and rebooting staging hosts after changes so problems show up before production.

Common mistakes

- Editing fstab without running `mount -a` to test it
- Leaving the journal non-persistent, so crash logs disappear
- Changing sysctl values with `-w` and forgetting to persist them
- Updating the kernel without keeping a previous one to fall back to
- Panicking and rebooting repeatedly instead of reading the previous boot's log

MARGA by EknathaLabs marga.eknathalabs.com

Check yourself

What starts after the kernel finishes initialising?

Answer: systemd as PID 1

Which command shows errors from the previous boot?

Answer: journalctl -b -1 -p err

How do you make a sysctl setting survive reboot?

Answer: Add it to a file in /etc/sysctl.d/ and run sysctl --system

What is a safe habit after editing /etc/fstab?

Answer: Run mount -a to confirm every entry mounts

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 12. Server hardening and auditing

16 minutes. Reduce a Linux server's attack surface with minimal packages, firewall rules, least privilege, SELinux or AppArmor, auditing and routine checks.

A new server is secure only if you make it so. Hardening means removing what you do not need, restricting what remains and watching for changes. Most breaches use known weaknesses: weak passwords, exposed services, unpatched software and excess privilege.

What you will learn

- Finding and closing unnecessary listening services
- A host firewall with a default-deny stance
- Mandatory access control with SELinux and AppArmor
- Auditing changes and scanning for weaknesses

Shrink the attack surface

Start by finding what is listening, then decide whether each service is needed on this host.

```
$ sudo ss -tulpn
$ systemctl list-unit-files --state=enabled --type=service | head -20
$ sudo systemctl disable --now cups avahi-daemon
$ dpkg -l | wc -l
$ sudo apt purge -y telnet rsh-client
```

Every listening port is an entry point. If a database should only be reached from the app tier, it should bind to a private interface (`bind-address = 10.0.1.5`), not `0.0.0.0`.

A default-deny firewall

```
$ sudo ufw default deny incoming
$ sudo ufw default allow outgoing
$ sudo ufw allow from 203.0.113.0/24 to any port 22 proto tcp
$ sudo ufw allow 443/tcp
$ sudo ufw enable
$ sudo ufw status numbered
```

On RHEL-family systems the same idea uses `firewalld`:

```
$ sudo firewall-cmd --permanent --add-service=https
$ sudo firewall-cmd --permanent --add-rich-rule='rule family="ipv4" source address="203.0.113.0/24 \
    " service name="ssh" accept'
$ sudo firewall-cmd --reload
$ sudo firewall-cmd --list-all
```

Cloud security groups are the first layer and a host firewall the second. Do both.

Least privilege

- Give each application its own non-login user and keep files owned by it
- Use `sudo` rules for specific commands, never blanket `ALL`
- Remove the setuid bit from binaries that do not need it

```
$ sudo visudo -f /etc/sudoers.d/deploy
$ sudo find / -xdev -perm -4000 -type f 2>/dev/null
$ sudo find / -xdev -type f -perm -0002 2>/dev/null | head
$ getent passwd | awk -F: '$7 !~ /nologin|false/ {print $1}'
```

A drop-in sudoers line such as `deploy ALL=(root) NOPASSWD: /bin/systemctl restart myapp` lets the deploy user restart one service and nothing else.

MARGA by EknathaLabs marga.eknathalabs.com

SELinux and AppArmor

Standard permissions answer "does this user own the file". **Mandatory access control** also asks "should this program touch this resource at all". RHEL-family systems use SELinux, Ubuntu uses AppArmor.

```
$ getenforce
$ sudo ausearch -m avc -ts recent | tail -5
$ sudo semanage port -a -t http_port_t -p tcp 8081
$ sudo restorecon -Rv /srv/www
$ sudo aa-status | head
```

When SELinux blocks something, **do not disable it**. Read the denial, then fix the file label or add the specific port or boolean the application legitimately needs. Disabling the control removes a safety net that has stopped real attacks.

Audit and verify

```
$ sudo apt install -y auditd lynis
$ sudo auditctl -w /etc/passwd -p wa -k identity
$ sudo ausearch -k identity | tail -3
$ sudo lynis audit system | tail -25
$ last -n 10
$ sudo lastb | head
```

`auditd` records who changed watched files. `lynis` and CIS benchmarks give a prioritised checklist. Ship audit logs off the host, because an attacker's first move is to edit local logs.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Server hardening and auditing fits together

Part	What it does	What goes wrong
Shrink the attack surface	Start by finding what is listening, then decide whether each service is needed on this host.	Shared accounts, and sudo rules that grant everything.
A default-deny firewall	On RHEL-family systems the same idea uses firewalld.	Disabling SELinux or the firewall to "fix" a problem.
Least privilege	Give each application its own non-login user and keep files owned by it.	Hardening a server once, then never re-checking after changes.
SELinux and AppArmor	Standard permissions answer "does this user own the file".	When SELinux blocks something, do not disable it.
Audit and verify	auditd records who changed watched files.	Keeping audit logs only on the server being attacked.

Flow: Shrink the attack surface to A default-deny firewall / A default-deny firewall to Least privilege / Least privilege to SELinux and AppArmor / SELinux and AppArmor to Audit and verify

Incident: the forgotten test database

> **Incident:** *A security scan finds MongoDB answering on a public IP with no authentication. It was started for a quick test months ago with `--bind_ip_all`, and the security group allowed all traffic. Customer records are exposed.*

Contain first: close the exposure, preserve evidence, then investigate.

```
$ sudo ss -tlnp | grep 27017
$ sudo ufw deny 27017
$ sudo systemctl stop mongod
$ sudo journalctl -u mongod --since "30 days ago" | grep -i "connection accepted" | head
```

Review connection logs for unknown sources, rotate any credentials stored in the database, and notify according to your policy. Prevent a repeat with an inventory of listening services, default-deny security groups, and automated scans that alert on any port open to the internet.

MARGA by EknathaLabs marga.eknathalabs.com

Common mistakes

- Disabling SELinux or the firewall to "fix" a problem
- Binding databases and caches to all interfaces
- Shared accounts, and sudo rules that grant everything
- Hardening a server once, then never re-checking after changes
- Keeping audit logs only on the server being attacked

Check yourself

What is the first step in reducing a server's attack surface?

Answer: List listening services and remove or restrict what is not needed

What does a default-deny firewall policy mean?

Answer: Incoming traffic is blocked unless a rule explicitly allows it

What should you do when SELinux blocks an application?

Answer: Read the denial and adjust the label, boolean or port the app needs

Why ship audit logs off the host?

Answer: An attacker who compromises the host can alter local logs

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 13. Namespaces and cgroups: what a container really is

15 minutes. Understand the two kernel features behind every container, namespaces that limit what a process can see and cgroups that limit what it can use, and inspect them with plain Linux tools.

A container is not a small virtual machine. It is an ordinary Linux process that the kernel has been told to treat in two special ways: it sees a restricted view of the system, and it may use only a limited amount of resources. Both features are in the kernel itself, and you can use them without Docker. Knowing this makes container behaviour, limits and escapes much easier to reason about.

What you will learn

- The main namespace types and what each isolates
- How cgroups limit CPU, memory and process counts
- How to look at both for any running process
- Why memory limits kill processes, and why "container root" is still root

Namespaces: a restricted view

Namespace	Isolates	Effect
pid	Process ids	The first process inside is pid 1 and cannot see host processes
net	Network devices, addresses, routes	Its own interfaces and ports
mnt	Mount points	Its own filesystem view
uts	Hostname	Its own hostname
ipc	Shared memory and queues	No sharing with other groups
user	User and group ids	Root inside can be a normal user outside
cgroup	The cgroup tree	Hides the host's resource layout

```
$ sudo unshare --pid --fork --mount-proc --uts bash
# hostname lab && ps aux
USER  PID %CPU %MEM COMMAND
root    1  0.0  0.1 bash
root    8  0.0  0.0 ps aux
```

Inside the new shell only two processes exist, because the pid namespace hides the rest. Every process has links to its namespaces in `/proc`:

```
ls -l /proc/$$/ns            # namespace ids of the current shell
sudo nsenter -t 4321 -n ip a # run `ip a` inside the network namespace of process 4321
lsns                         # list namespaces on the host
```

`nsenter` is the way to debug a container that has no tools: run the host's tools inside the container's namespaces.

cgroups: a limited budget

Control groups account for and limit resources of a set of processes. On modern systems (cgroup v2) they appear as files:

```
cat /sys/fs/cgroup/system.slice/shop.service/memory.max     # memory limit in bytes
cat /sys/fs/cgroup/system.slice/shop.service/memory.current # use right now
cat /sys/fs/cgroup/system.slice/shop.service/cpu.max        # quota and period, e.g. 50000 100000  \
    = half a CPU
cat /sys/fs/cgroup/system.slice/shop.service/pids.max
```

Memory is a hard limit: when a group goes over `memory.max` and cannot free memory, the kernel kills a process in it (an OOM kill, exit code 137). CPU is a throttle: a group over its quota is paused until the next

period, which shows up as latency, not as a crash. Check `cpu.stat` for `nr_throttled`.

systemd uses cgroups too: `systemctl set-property shop.service MemoryMax=512M CPUQuota=50%` sets limits on a service with no containers at all.

What this means in practice

- A container shares the host kernel, so a kernel bug affects every container. This is the main difference from a virtual machine.
- Root in a container is real root unless a user namespace maps it. Drop capabilities, run as non-root, and use seccomp profiles.
- Tools inside a container may show the host's memory or CPU count, not the limit, so applications must read cgroup values to size themselves.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Namespaces and cgroups: what a container really is fits together

Part	What it does	What goes wrong
Namespaces: a restricted view	Inside the new shell only two processes exist, because the pid namespace hides the rest.	Reading free inside a container and trusting it as the limit.
cgroups: a limited budget	Control groups account for and limit resources of a set of processes.	Setting a memory limit equal to the heap size.
What this means in practice	A container shares the host kernel, so a kernel bug affects every container.	Thinking a container is a virtual machine.

Flow: Namespaces: a restricted view to cgroups: a limited budget / cgroups: a limited budget to What this means in practice

Incident: the Java service that kept dying

Incident: *A service in a container with a 1 GB limit is killed every few hours with exit code 137, while its own heap setting looks safe.*

```
$ cat /sys/fs/cgroup/memory.max
1073741824
$ cat /sys/fs/cgroup/memory.events
oom_kill 7
```

The runtime saw the host's 64 GB and sized its heap and thread stacks for that, so total process memory (heap plus native memory) went past 1 GB. The team set the heap as a percentage of the container limit, left room for native memory, and alerted on `memory.current` approaching `memory.max`.

Common mistakes

- Thinking a container is a virtual machine
- Reading `free` inside a container and trusting it as the limit
- Setting a memory limit equal to the heap size
- Giving containers capabilities "to make it work"

Check yourself

What does a pid namespace do?

Answer: Gives a process group its own view of process ids

Which kernel feature limits how much memory a container may use?

Answer: cgroups

MARGA by EknathaLabs marga.eknathalabs.com

What happens when a cgroup exceeds its CPU quota?

Answer: It is paused until the next period, so it runs slower

How can you run host tools inside a container's network namespace?

Answer: nsenter -t PID -n command

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 14. Working efficiently in the terminal

12 minutes. Move faster and make fewer mistakes with shell history, keyboard shortcuts, tmux sessions, a safe editor setup, aliases and dotfiles you can carry between machines.

Operations work happens in a terminal, often over a slow connection and often under stress. Small habits decide whether you type a command twice or ten times, lose a session when the network drops, or run a destructive command in the wrong place. This lesson collects the habits that pay off most.

What you will learn

- Shell history and line-editing shortcuts
- Persistent sessions with `tmux`
- Basic editing in `vim` or `nano` over SSH
- Aliases, functions and a prompt that prevents mistakes
- Keeping your setup in dotfiles

History and shortcuts

Keys	Action
`Ctrl+R`	Search history backwards while typing
`Ctrl+A` / `Ctrl+E`	Start or end of the line
`Ctrl+W`	Delete the previous word
`Alt+.`	Insert the last argument of the previous command
`!!`	Repeat the previous command (`sudo !!` runs it with sudo)
`Ctrl+L`	Clear the screen

```
HISTSIZE=50000 HISTFILESIZE=100000
HISTCONTROL=ignoreboth:erasedups   # skip duplicates and commands starting with a space
shopt -s histappend                # keep history from all open shells
```

A command starting with a space is not saved, which is useful when a secret appears on the command line (better still: avoid secrets on command lines).

Sessions that survive: tmux

```
tmux new -s fix          # start a named session
# Ctrl+B then d          # detach, the session keeps running
tmux attach -t fix       # come back later, even from another connection
# Ctrl+B then %          # split left and right
# Ctrl+B then "          # split top and bottom
```

A long job started inside `tmux` is not lost if your connection drops, and a colleague can attach to the same session to watch or help.

Editing on a server

Learn the few keys of one editor that exists everywhere. In `vim`: `i` to insert, `Esc` to leave insert mode, `:w` save, `:q` quit, `:q!` quit without saving, `/word` search, `dd` delete a line, `u` undo. If you prefer, `nano` shows its keys at the bottom. Edit with `sudoedit /etc/file` rather than `sudo vim`, so the editor runs as you and the change is saved safely.

Aliases, functions and a safer prompt

```
alias ll='ls -lah'
alias k=kubectl
mkcd() { mkdir -p "$1" && cd "$1"; }
export PS1='\u@\h:\w\$ '    # show user, host and directory
```

MARGA by EknathaLabs marga.eknathalabs.com

Show the host name clearly, and use a different colour for production hosts, so you notice before running a destructive command. For Kubernetes, show the current context in the prompt for the same reason.

Dotfiles

Keep `.bashrc`, `.vimrc`, `.tmux.conf` and similar files in one directory under version control, and install them with a small script that creates symbolic links. A new machine then feels familiar in minutes. Keep secrets out of dotfiles.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Working efficiently in the terminal fits together

Part	What it does	What goes wrong
History and shortcuts	A command starting with a space is not saved, which is useful when a secret appears on the command line (better still: avoid secrets on command lines).	Secrets typed on the command line and kept in history.
Sessions that survive: tmux	A long job started inside tmux is not lost if your connection drops, and a colleague can attach to the same session to watch or help.	Long jobs run without tmux or nohup.
Editing on a server	Learn the few keys of one editor that exists everywhere.	Editing system files as root in a full editor session.
Aliases, functions and a safer	Show the host name clearly, and use a different colour for production hosts, so you notice before running a destructive command.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Dotfiles	Keep .bashrc, .vimrc, .tmux.conf and similar files in one directory under version control, and install them with a small script that creates symbolic links.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: History and shortcuts to Sessions that survive: tmux / Sessions that survive: tmux to Editing on a server / Editing on a server to Aliases, functions and a safer / Aliases, functions and a safer to Dotfiles

Incident: the wrong host

Incident: *An engineer runs* `rm -rf /var/lib/app/cache` *to free space, in a terminal that was connected to production instead of staging.*

```
$ hostname
prod-web-03
```

Both terminals looked the same. The team set a red background and a clear host name in the prompt for production, added a confirmation wrapper for destructive commands on production hosts, and used `tmux` window names such as "STAGING" and "PROD".

Common mistakes

- Long jobs run without `tmux` or `nohup`
- Identical-looking prompts for every environment
- Editing system files as root in a full editor session
- Secrets typed on the command line and kept in history

Check yourself

Which key combination searches shell history while you type?

Answer: Ctrl+R

Why use tmux on a remote server?

MARGA by EknathaLabs marga.eknathalabs.com

Answer: A session keeps running when your connection drops, and you can reattach

What does sudoedit do better than sudo vim?

Answer: It edits a copy as your user and saves it with privileges, avoiding a root editor session

How can you reduce the chance of running a command on the wrong host?

Answer: Make the prompt show the host and mark production clearly

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 15. Time, NTP and time zones

12 minutes. Keep server clocks correct with NTP, understand UTC versus local time, and diagnose failures that come from clock drift.

A login token is rejected as "not yet valid", a certificate seems to expire a day early, and logs from two servers cannot be lined up. All three problems have the same cause: the servers do not agree on the time. Time looks like a small topic until it breaks authentication, replication and every investigation that relies on timestamps.

What you will learn

- The difference between the hardware clock, the system clock and a time zone
- How NTP keeps clocks in step and how to check that it works
- Why servers should log and run in UTC
- How clock drift breaks tokens, certificates and logs

Clocks and zones

Linux keeps a **system clock** that counts time since 1970 in UTC. A **time zone** only decides how that number is shown to people. The **hardware clock** (RTC) keeps time while the machine is off, and is read at boot.

```
$ date
Tue Jan  6 10:02:11 UTC 2026
$ timedatectl
               Local time: Tue 2026-01-06 10:02:11 UTC
           Universal time: Tue 2026-01-06 10:02:11 UTC
                 Time zone: Etc/UTC (UTC, +0000)
System clock synchronized: yes
              NTP service: active
```

The two lines to watch are `System clock synchronized` and `NTP service`. Set the zone with `sudo timedatectl set-timezone Etc/UTC`. For servers, UTC is the best choice: no daylight saving changes, and logs from every region sort in one order.

How NTP keeps time

The **Network Time Protocol** asks several time servers for the time, estimates the network delay, and adjusts the local clock. Small errors are corrected by *slewing*, which runs the clock slightly faster or slower. Large errors may cause a *step*, a sudden jump, which can upset software that measures durations.

On most distributions the client is `systemd-timesyncd` or `chrony`. Chrony is the usual choice on servers because it recovers quickly and works well on virtual machines.

```
$ chronyc tracking
Reference ID    : A9FEA97B (169.254.169.123)
Stratum         : 4
System time     : 0.000012345 seconds fast of NTP time
Last offset     : +0.000008123 seconds
$ chronyc sources -v
```

The `System time` line is the current error. Milliseconds are normal; seconds are a problem. In a cloud, prefer the provider's own time source (for example the link-local address shown above), which is close and reliable.

What breaks when time is wrong

Many systems trust the clock more than you might expect:

Feature	What goes wrong with a wrong clock
TLS certificates	A certificate looks "not yet valid" or "expired"
Signed tokens (JWT, cloud API signatures)	Requests rejected, often with a clock skew message

MARGA by EknathaLabs marga.eknathalabs.com

Feature	What goes wrong with a wrong clock
Kerberos	Logins fail when skew is more than a few minutes
Databases and clusters	Replication ordering and leases misbehave
Logs and metrics	Events from two hosts appear in the wrong order

A cloud API that says the signature has expired, when the key is correct, is a classic sign of drift of more than five minutes.

Fixing and checking drift

Check first, then fix the cause, not only the symptom.

```
timedatectl status
chronyc tracking
sudo systemctl enable --now chrony
sudo chronyc makestep      # step the clock once, now
```

If a virtual machine drifts after it was paused or migrated, enable the guest time-sync feature or point chrony at the hypervisor or cloud time source. Never set the clock by hand on a production server as a lasting fix. It hides the problem and the drift returns. Add a monitoring alert on the offset (for example, over 100 ms) so that you hear about drift before users do.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Time, NTP and time zones fits together

Part	What it does	What goes wrong
Clocks and zones	Linux keeps a system clock that counts time since 1970 in UTC.	Using local time with daylight saving on servers, so logs repeat or skip an hour.
How NTP keeps time	The Network Time Protocol asks several time servers for the time, estimates the network delay, and adjusts the local clock.	Forgetting that a restored or paused virtual machine may start with an old clock.
What breaks when time is wrong	Many systems trust the clock more than you might expect.	Setting the clock by hand and not fixing the time service.
Fixing and checking drift	Check first, then fix the cause, not only the symptom.	Blocking UDP port 123 on a firewall and not noticing for months.

Flow: Clocks and zones to How NTP keeps time / How NTP keeps time to What breaks when time is wrong / What breaks when time is wrong to Fixing and checking drift

Incident: Tokens rejected after a restore

Incident: *After a virtual machine was restored from a snapshot, every call from it to the cloud API fails with "signature expired", although the credentials are valid.*

```
$ date -u
Mon Jan  5 08:15:40 UTC 2026
$ curl -sI https://api.example.com | grep -i ^date
date: Tue, 06 Jan 2026 10:02:05 GMT
$ chronyc tracking | grep "System time"
System time     : 93815.2 seconds slow of NTP time
```

The restored machine woke up with the time of the snapshot, a day behind, and the time client was disabled in the image. Cloud APIs reject signatures that differ from their own clock by more than a few minutes. The fix was `chronyc makestep` and enabling the chrony service, and the team added a start-up check that waits for synchronization before the application starts, plus an alert on clock offset.

Common mistakes

- Setting the clock by hand and not fixing the time service

MARGA by EknathaLabs marga.eknathalabs.com

- Using local time with daylight saving on servers, so logs repeat or skip an hour
- Forgetting that a restored or paused virtual machine may start with an old clock
- Blocking UDP port 123 on a firewall and not noticing for months

Check yourself

Which time zone is the best default for servers?

Answer: UTC, so logs from all regions line up and there are no daylight saving jumps

What does NTP do?

Answer: Adjusts the system clock using time servers, correcting errors by slewing or stepping

Which symptom most suggests clock drift?

Answer: Valid credentials rejected with a signature expired or not yet valid message

Which port does NTP use?

Answer: UDP 123

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 16. systemd in depth: overrides, targets and dependencies

14 minutes. Change a service safely with drop-in overrides, understand ordering and dependencies, and use targets and resource settings.

You need one extra environment variable for a packaged service. Editing the file under /usr/lib works until the next package update replaces it, and then the service quietly loses the setting. systemd has a clean answer, the drop-in override, and a few concepts that explain why services start in the order they do.

What you will learn

- Edit a unit safely with `systemctl edit` and drop-in files
- Order and dependencies: After, Requires, Wants
- Targets and what happens at boot
- Restart policies and resource limits in a unit

Where unit files live and who wins

Packages install units in `/usr/lib/systemd/system/`. Your changes belong in `/etc/systemd/system/`, which has priority and is never touched by package updates.

```
$ systemctl cat shop.service
# /usr/lib/systemd/system/shop.service
[Service]
ExecStart=/usr/bin/shop
$ sudo systemctl edit shop.service
```

`systemctl edit` opens an empty file at `/etc/systemd/system/shop.service.d/override.conf`. Put only the lines you want to change there:

```
[Service]
Environment=SHOP_MODE=production
Restart=on-failure
RestartSec=5
```

After saving, systemd reloads the configuration for you. To clear a list setting such as `ExecStart`, set it to empty first, then set the new value. `systemctl cat` shows the original and all drop-ins together, which is the first thing to read when a service behaves unexpectedly.

Ordering is not the same as dependency

Two ideas are often confused:

- **Ordering** (`After=`, `Before=`) says which unit starts first if both are being started.
- **Dependency** (`Requires=`, `Wants=`) says that starting this unit pulls in another.

`Wants=` is a soft dependency: if the other unit fails, this one still starts. `Requires=` is hard: failure of the other stops this unit. A common, correct pair is `Wants=network-online.target` together with `After=network-online.target`, so the service starts after the network is really up.

```
[Unit]
Description=Shop API
Wants=network-online.target postgresql.service
After=network-online.target postgresql.service
```

If you write only `After=` the other unit is not started for you, and if you write only `Wants=` the two may start at the same moment. You usually need both.

MARGA by EknathaLabs
marga.eknathalabs.com

Targets and boot

A **target** is a named group of units, a goal for boot. `multi-user.target` is the usual server state (no graphical session), and `graphical.target` adds a desktop. `systemctl enable` makes a symlink so that a unit is pulled in by a target, usually `multi-user.target`, through the `WantedBy=` line in `[Install]`.

```
$ systemctl get-default
multi-user.target
$ systemctl list-dependencies multi-user.target | head
$ systemd-analyze blame | head -5
$ systemd-analyze critical-chain
```

`systemd-analyze blame` shows which units took the most time to start, and `critical-chain` shows the chain that decided the total boot time. Use them when a server boots slowly.

Restarts and limits

A unit can describe how it should behave when things go wrong, so that you do not need a separate watchdog script.

```
[Service]
Restart=on-failure
RestartSec=5
StartLimitIntervalSec=300
StartLimitBurst=5
MemoryMax=1G
CPUQuota=150%
LimitNOFILE=65536
ProtectSystem=strict
NoNewPrivileges=true
```

`StartLimit*` stops a crash loop after five tries in five minutes, and the unit then shows `failed` instead of hiding the problem. `MemoryMax` and `CPUQuota` use cgroups, so one service cannot starve the others. The `Protect*` and `NoNewPrivileges` lines reduce what a compromised service can do, and `systemd-analyze security shop.service` scores them.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How systemd in depth: overrides, targets and... fits together

Part	What it does	What goes wrong
Where unit files live and who wins	Packages install units in /usr/lib/systemd/system/.	Editing files in /usr/lib/systemd/system instead of using an override.
Ordering is not the same as	Two ideas are often confused.	Writing After= without Wants=, so the dependency never starts.
Targets and boot	A target is a named group of units, a goal for boot.	Using Restart=always with no start limit, which hides crash loops.
Restarts and limits	A unit can describe how it should behave when things go wrong, so that you do not need a separate watchdog script.	Forgetting daemon-reload after a manual edit.

Flow: Where unit files live and who wins to Ordering is not the same as / Ordering is not the same as to Targets and boot / Targets and boot to Restarts and limits

Incident: The setting that vanished

> **Incident:** *After a routine package update, the shop service started failing at 03:00 because a database password variable was missing.*

```
$ systemctl cat shop.service | head -4
# /usr/lib/systemd/system/shop.service
[Service]
ExecStart=/usr/bin/shop
$ ls /etc/systemd/system/shop.service.d/
```

MARGA by EknathaLabs marga.eknathalabs.com

```
ls: cannot access ...: No such file or directory
```

Someone had added `Environment=` lines straight into the packaged file under /usr/lib. The package update replaced the file, and the variable was gone. The fix was a drop-in override in /etc, plus an `EnvironmentFile=` with strict permissions for the secret. The team also added a check to the weekly review that lists services with local changes by running systemd-delta.

Common mistakes

- Editing files in /usr/lib/systemd/system instead of using an override
- Writing After= without Wants=, so the dependency never starts
- Forgetting daemon-reload after a manual edit
- Using Restart=always with no start limit, which hides crash loops

Check yourself

Where should local changes to a packaged unit go?

Answer: A drop-in file under /etc/systemd/system/NAME.service.d/

What does After=network-online.target do on its own?

Answer: Orders this unit after the target, but does not start the target

Which command shows which unit slowed the boot most?

Answer: systemd-analyze blame

What is the benefit of StartLimitBurst?

Answer: A unit that keeps crashing stops after a few tries and shows as failed

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 17. Kernel modules, sysctl and boot parameters

13 minutes. Load and inspect kernel modules, tune the running kernel with sysctl, set boot parameters, and read the kernel ring buffer.

The kernel is not a single fixed program. Drivers and features can be loaded as modules, behaviour is tuned through thousands of runtime parameters, and some settings can only be given on the boot command line. Engineers who can read and change these safely solve problems that no application setting can touch, such as a connection table that fills up or a driver that misbehaves after an update.

What you will learn

- List, load and block kernel modules
- Read and change runtime parameters with sysctl
- Persist settings and understand the boot command line
- Read kernel messages with dmesg and journalctl -k

Modules

A module is a piece of kernel code loaded on demand, such as a filesystem, a network driver or the module that implements overlay mounts for containers.

```
lsmod | head
modinfo overlay
sudo modprobe overlay
sudo modprobe -r overlay
lsmod | grep -E "overlay|br_netfilter"
```

`lsmod` lists what is loaded, `modinfo` shows the module description and parameters, and `modprobe` loads a module with its dependencies (unlike `insmod`, which loads one file). To load a module at every boot, list it in a file under `/etc/modules-load.d/`. To prevent one from loading, add `blacklist name` or `install name /bin/false` under `/etc/modprobe.d/`. Remember that modules run with full kernel privileges: load only what you trust. With Secure Boot, modules must be signed.

sysctl: runtime tuning

Kernel parameters appear as files under `/proc/sys` and are read and written with `sysctl`.

```
sysctl net.ipv4.ip_forward
sudo sysctl -w net.ipv4.ip_forward=1
sysctl -a 2>/dev/null | grep -c .
sudo sysctl --system
```

`-w` changes the running kernel only, and the change disappears at reboot. To keep it, put the line in a file such as `/etc/sysctl.d/90-shop.conf` and run `sysctl --system`. Parameters you will meet often: `net.ipv4.ip_forward` (routing and container networking), `vm.swappiness` (how readily memory is swapped), `vm.max_map_count` (needed by some search engines), `fs.file-max` and `fs.inotify.max_user_watches` (many files or watchers), `net.core.somaxconn` (listen queue length) and `net.ipv4.tcp_*` settings. Change one at a time, record the old value, and measure before and after.

Boot parameters

Some settings are read once, at boot, from the kernel command line. Look at the current one:

```
cat /proc/cmdline
BOOT_IMAGE=/vmlinuz-6.8.0 root=UUID=7a1c... ro quiet splash console=ttyS0 transparent_hugepage=mad \
    vise
```

Typical parameters are `root=` (the root filesystem), `console=` (where messages go), `quiet`, `nomodeset`, `systemd.unit=rescue.target` (boot into rescue mode), `init=/bin/bash` (emergency shell) and `mem=`. On most distributions you change defaults in `/etc/default/grub` (the `GRUB_CMDLINE_LINUX` line) and regenerate the

MARGA by EknathaLabs
marga.eknathalabs.com

config with `update-grub` or `grub2-mkconfig`. For a one-time test, press `e` in the boot menu and edit the line; nothing is saved. Keep a known good entry available before changing anything.

Kernel messages

```
dmesg -T --level=err,warn | tail
journalctl -k -b -1 | tail -50      # kernel messages of the previous boot
journalctl -k -p err -b
```

The kernel keeps a ring buffer of messages: hardware detection, driver errors, filesystem problems, out-of-memory kills and network warnings such as `nf_conntrack: table full, dropping packet`. Read it first when behaviour looks impossible from user space. `-T` prints readable times; the previous boot's messages require persistent journals. Messages from before a crash are often the only evidence, so make sure `/var/log/journal` exists on servers.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Kernel modules, sysctl and boot parameters fits together

Part	What it does	What goes wrong
Modules	A module is a piece of kernel code loaded on demand, such as a filesystem, a network driver or the module that implements overlay mounts for containers.	Loading unsigned or unknown modules on production servers.
sysctl: runtime tuning	Kernel parameters appear as files under /proc/sys and are read and written with sysctl.	Changing many parameters at once.
Boot parameters	Some settings are read once, at boot, from the kernel command line.	Editing the boot command line without a fallback entry.
Kernel messages	The kernel keeps a ring buffer of messages: hardware detection, driver errors, filesystem problems, out-of-memory kills and network warnings such as nf_conntrack: table...	Using sysctl -w and forgetting to persist it.

Flow: Modules to sysctl: runtime tuning / sysctl: runtime tuning to Boot parameters / Boot parameters to Kernel messages

Incident: Packets dropped, nothing in the application logs

Incident: *A gateway dropped new connections during peak hours while CPU was low.*

```
$ dmesg -T | tail -3
[Tue Jan  6 18:03:11 2026] nf_conntrack: nf_conntrack: table full, dropping packet
[Tue Jan  6 18:03:12 2026] nf_conntrack: nf_conntrack: table full, dropping packet
$ sysctl net.netfilter.nf_conntrack_max net.netfilter.nf_conntrack_count
net.netfilter.nf_conntrack_max = 65536
net.netfilter.nf_conntrack_count = 65531
```

The connection tracking table was full, so the kernel dropped new connections before the application saw them. The team raised nf_conntrack_max in a file under /etc/sysctl.d/ after checking memory use, shortened some timeouts, and added an alert on the count to max ratio. The lesson: some limits exist only in the kernel, and dmesg is where they speak.

Common mistakes

- Using sysctl -w and forgetting to persist it
- Changing many parameters at once
- Editing the boot command line without a fallback entry
- Loading unsigned or unknown modules on production servers

MARGA by EknathaLabs marga.eknathalabs.com

Check yourself

How do you load a module with its dependencies?

Answer: modprobe

Where do you persist a sysctl setting?

Answer: A file under /etc/sysctl.d/ followed by sysctl --system

How do you see the boot parameters in use?

Answer: cat /proc/cmdline

Where do you find kernel warnings such as a full conntrack table?

Answer: dmesg or journalctl -k

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 18. Memory, swap and the OOM killer

14 minutes. Understand virtual memory, the page cache and swap, read free and /proc/meminfo correctly, and investigate an out-of-memory kill.

Memory problems look mysterious because Linux uses memory differently from how people expect. Free memory near zero is normal, since the kernel uses spare memory as cache. A process can be killed even when free shows gigabytes available, if its cgroup limit was hit. Knowing what the numbers mean, and where the kernel records its decisions, turns an out-of-memory incident from guessing into reading.

What you will learn

- Virtual memory, resident memory and the page cache
- Read free, /proc/meminfo and per-process numbers
- How the OOM killer chooses a victim and where it logs
- Container memory limits and how to prevent kills

What the numbers mean

```
free -h
               total   used   free  shared  buff/cache  available
Mem:            15Gi   4.1Gi  0.4Gi  0.2Gi       11Gi       10Gi
Swap:          2.0Gi   0.0Gi  2.0Gi
```

The `free` column is memory that nothing uses; `buff/cache` is file data kept in memory to speed up reads, which the kernel gives back when programs need it. The column that matters is **available**: an estimate of how much can be given to programs without swapping. Each process has **virtual** memory (address space promised to it, often much larger than real use), **resident** memory (RSS, pages actually in RAM) and shared pages counted in several processes. Use `ps -eo pid,rss,cmd --sort=-rss | head` and `/proc/PID/smaps_rollup` for accurate per-process values.

Swap and pressure

Swap is disk space the kernel can use to move rarely used pages out of RAM. A little swap use is fine. A system that constantly reads and writes swap is **thrashing**: it is slow, with high I/O wait and low CPU use.

```
vmstat 1 5            # watch si/so (swap in/out) and wa (I/O wait)
cat /proc/pressure/memory
sysctl vm.swappiness
```

The pressure stall information (PSI) file shows how much time tasks spend waiting for memory: values like `some avg10=12.5` mean tasks were stalled 12.5 percent of the last ten seconds. PSI is a better alarm than free memory. Many container hosts run with no swap on purpose, so that overload shows up as a clear kill rather than as a slow death.

The OOM killer

When the kernel cannot find memory, it picks a process to kill so that the system can survive. It prefers processes that use a lot of memory, adjusted by `oom_score_adj` (range -1000 to 1000; -1000 makes a process unkillable, so use it carefully).

```
dmesg -T | grep -i -E "out of memory|killed process"
journalctl -k | grep -i oom
cat /proc/$(pidof myapp)/oom_score
```

A typical message reads: `Out of memory: Killed process 4121 (java) total-vm:9876540kB, anon-rss:7340000kB`. The log lists the victim, its sizes and a table of all processes. Check whether the kill came from the whole machine running out, or from a **cgroup** limit (the message mentions `Memory cgroup out of memory`).

MARGA by EknathaLabs marga.eknathalabs.com

Containers and prevention

In containers and Kubernetes pods, the memory limit becomes a cgroup limit, and exceeding it gets the container killed (status `OOMKilled`, exit code 137) even when the host has plenty. Prevention is design work:

- Set requests and limits from measured usage, with headroom.
- Make runtimes aware of the limit (for example, set the heap as a percentage of the container memory).
- Fix leaks and unbounded caches or queues.
- Alert on memory working set and on PSI, not only on free memory.
- Keep important system daemons protected with a lower `oom_score_adj`.

```
kubectl describe pod web-7d9 | grep -A3 "Last State"
kubectl top pod web-7d9 --containers
```

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Memory, swap and the OOM killer fits together

Part	What it does	What goes wrong
What the numbers mean	The free column is memory that nothing uses; buff/cache is file data kept in memory to speed up reads, which the kernel gives back when programs need it.	Reading the free column as the amount of memory left.
Swap and pressure	Swap is disk space the kernel can use to move rarely used pages out of RAM.	Disabling swap or the OOM killer without a plan.
The OOM killer	When the kernel cannot find memory, it picks a process to kill so that the system can survive.	Giving a process oom_score_adj of -1000 without thought.
Containers and prevention	In containers and Kubernetes pods, the memory limit becomes a cgroup limit, and exceeding it gets the container killed (status OOMKilled, exit code 137) even when the...	Setting container limits below the real working set.

Flow: What the numbers mean to Swap and pressure / Swap and pressure to The OOM killer / The OOM killer to Containers and prevention

Incident: The pod killed with plenty of host memory

Incident: *A Java service restarted every few hours with exit code 137.*

```
$ kubectl describe pod orders-6f7 | grep -A4 "Last State"
    Last State:   Terminated
      Reason:     OOMKilled
      Exit Code:  137
$ kubectl get pod orders-6f7 -o jsonpath='{.spec.containers[0].resources.limits.memory}'
512Mi
```

The JVM was configured with a fixed 1 GB heap while the container limit was 512 MiB, so the kernel killed it inside its cgroup when the heap grew. The fix set the heap as a percentage of the container memory, raised the limit based on measurements, and added an alert on working set above 85 percent of the limit.

Common mistakes

- Reading the free column as the amount of memory left
- Disabling swap or the OOM killer without a plan
- Setting container limits below the real working set
- Giving a process oom_score_adj of -1000 without thought

MARGA by EknathaLabs marga.eknathalabs.com

Check yourself

Which column of free best shows memory available to programs?

Answer: available

What does exit code 137 with OOMKilled usually mean?

Answer: The container was killed after exceeding its memory limit

Where do you find which process the kernel killed?

Answer: dmesg or journalctl -k

Why is thrashing slow?

Answer: The system constantly moves pages between RAM and swap on disk

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 19. System calls, strace and perf

14 minutes. See how programs talk to the kernel through system calls, trace them with strace, and find hot spots with perf.

Every program that touches a file, a socket or the clock must ask the kernel through a system call. When an application hangs, fails with a vague message or burns CPU for no clear reason, watching its system calls shows what it is really doing, regardless of the language it is written in. Two tools do most of the work: strace for the conversation with the kernel and perf for where the time goes.

What you will learn

- What a system call is and what user and kernel space mean
- Trace a process with strace and read its output
- Find permission, missing file and DNS problems quickly
- Profile CPU use with perf and know its limits

User space and kernel space

Programs run in **user space** with limited rights. To read a file, open a connection or create a process they call into the kernel, which runs in **kernel space** with full access: `open`, `read`, `write`, `connect`, `accept`, `clone`, `execve`, `mmap`, `futex` and hundreds more. Libraries such as the C library wrap them. Switching between modes costs time, so a program making millions of tiny calls is slow, and a program stuck in one call (waiting for a read that never returns) appears hung. A list is in `man 2 syscalls`.

Using strace

```
strace -f -tt -T -o /tmp/trace.txt ./myapp --config /etc/myapp.conf
strace -p 4121 -f -e trace=network
strace -c -p 4121                   # summary table by call, then Ctrl-C
strace -e trace=openat,stat ls /nonexistent 2>&1 | tail -3
```

`-f` follows child processes and threads, `-tt` adds timestamps, `-T` shows the time spent inside each call, `-e trace=` filters by call or group, and `-c` prints a summary of counts and time. A line such as `openat(AT_FDCWD, "/etc/myapp.conf", O_RDONLY) = -1 ENOENT (No such file or directory)` tells you exactly which file was missing. The return value after `=` and the error name are the key.

Typical findings

- **Wrong file or permissions**: `ENOENT` or `EACCES` on `openat`; shows the exact path the program really tried.
- **Hang**: the last line is a call that has not returned, such as `read(7,` or `futex(0x7f...,` (waiting on a lock) or `connect(` (waiting on a network peer).
- **DNS or config**: reads of `/etc/resolv.conf` and a `sendto` to port 53 followed by a long wait.
- **Too many calls**: a flood of `stat` or `read` of 1 byte shows a bad loop; `strace -c` makes this visible.

strace slows the target a lot, because it stops it at every call. Use it on a test copy or briefly in production, and consider `perf trace` or eBPF-based tools for low-overhead tracing.

perf for CPU time

```
sudo perf top
sudo perf record -F 99 -g -p 4121 -- sleep 30
sudo perf report --stdio | head -40
```

perf samples what the CPU is doing 99 times per second and shows which functions use the most time, including kernel code. If a process uses 100 percent CPU and strace shows no calls, it is computing in user space, and perf shows where. Without debug symbols you see addresses instead of names. Flame graphs built from perf data are a common way to present the result. Run profiling during the problem, for a short period, and save the data with the incident notes.

MARGA by EknathaLabs
marga.eknathalabs.com

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How System calls, strace and perf fits together

Part	What it does	What goes wrong
User space and kernel space	Programs run in user space with limited rights.	Running strace on a busy production process for a long time.
Using strace	-f follows child processes and threads, -tt adds timestamps, -T shows the time spent inside each call, -e trace= filters by call or group, and -c prints a summary of...	Ignoring return values and error names.
Typical findings	Wrong file or permissions: ENOENT or EACCES on openat; shows the exact path the program really tried.	Forgetting -f and tracing only the parent.
perf for CPU time	perf samples what the CPU is doing 99 times per second and shows which functions use the most time, including kernel code.	Profiling without symbols or for too short a time.

Flow: User space and kernel space to Using strace / Using strace to Typical findings / Typical findings to perf for CPU time

Incident: The config file nobody could find

Incident: *An application started with default settings, ignoring the settings in its configuration file.*

```
$ strace -f -e trace=openat ./myapp 2>&1 | grep -i conf
openat(AT_FDCWD, "/etc/myapp/config.yaml", O_RDONLY) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/home/app/.myapp.yaml", O_RDONLY) = -1 ENOENT (No such file or directory)
```

The application looked for /etc/myapp/config.yaml, while the team had placed the file at /etc/myapp.yaml. A path mismatch after a packaging change was the cause. Tracing showed the exact paths in a minute, which saved reading the source code. The packaging was fixed and a startup check was added that fails loudly when no configuration is found.

Common mistakes

- Running strace on a busy production process for a long time
- Forgetting -f and tracing only the parent
- Ignoring return values and error names
- Profiling without symbols or for too short a time

Check yourself

What does -f do in strace?

Answer: Follows child processes and threads

What does ENOENT mean in a trace?

Answer: The path does not exist

Which tool finds the functions using CPU time?

Answer: perf

Why is strace risky on a busy production service?

Answer: It stops the process at each system call and slows it down

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 20. Filesystems, the page cache and I/O

14 minutes. Follow a read and a write through the VFS and page cache, understand writeback and fsync, and diagnose slow disks with iostat.

Disk behaviour confuses many engineers because the kernel sits in between. A write that returns immediately has not necessarily reached the disk. A read may never touch the disk at all. Understanding the virtual file system, the page cache and writeback explains why databases insist on fsync, why a copy seems instant then stalls, and how to tell a slow disk from a slow application.

What you will learn

- VFS and what happens on read and write
- Dirty pages, writeback and fsync
- Read iostat and find the saturated device
- Common filesystem limits: inodes, descriptors, mount options

The path of a read and a write

Programs call `read` and `write` on a file descriptor. The **VFS** (virtual file system) layer gives all filesystems (ext4, XFS, overlayfs, NFS) the same interface. Reads are served from the **page cache** if the data is there; otherwise the kernel reads blocks from the device and caches them. Writes go into the cache as **dirty pages** and return at once; the kernel writes them to the device later (**writeback**). That is fast, but data in dirty pages is lost if power fails before writeback.

fsync and durability

```
sync                                  # flush all dirty data now
grep -E "Dirty|Writeback" /proc/meminfo
sysctl vm.dirty_ratio vm.dirty_background_ratio
```

Applications that must not lose data call `fsync` (or open files with `O_SYNC`) so the kernel writes the file to stable storage before returning. Databases and message queues do this for their logs, which is why their speed depends on disk latency. If too much dirty data builds up, the kernel makes writers wait (`vm.dirty_ratio`), and you see long pauses during large copies. Filesystems also keep a journal so that metadata is consistent after a crash; a journal is not a backup.

Finding a slow disk

```
iostat -xz 1 3
Device  r/s   w/s  rkB/s  wkB/s  await  aqu-sz  %util
nvme0n1 120  3400  4800  98000   1.2    4.1    62
sda      15    80   400   2200  85.0    7.5    99
```

Columns to watch: `await` (average time per request including queueing, in ms), `aqu-sz` (queue length) and `%util` (how busy the device is; on SSDs and arrays near 100 percent does not always mean saturation, but high await does). In the example, `sda` has an average wait of 85 ms and is nearly fully busy: it is the bottleneck. Find the process responsible with `iotop -oa` or `pidstat -d 1`, and check high `wa` (I/O wait) in `vmstat` or `top`.

Limits that surprise people

- **Inodes**: `df -i` shows inode use; millions of tiny files can exhaust inodes while `df -h` shows free space.
- **Open files**: `ulimit -n` and `/proc/PID/limits` limit descriptors per process; `Too many open files` errors point here.
- **Mount options**: `noatime` reduces metadata writes; `ro` after errors means the filesystem was remounted read-only, so check `dmesg` for I/O errors.
- **Full volumes**: logs and container layers fill disks; `du -xh --max-depth=1 /var | sort -h` finds the biggest directories.

MARGA by EknathaLabs
marga.eknathalabs.com

- **Network filesystems**: a stuck NFS server leaves processes in uninterruptible sleep (state `D`) that cannot be killed.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Filesystems, the page cache and I/O fits together

Part	What it does	What goes wrong
The path of a read and a write	Programs call read and write on a file descriptor.	Assuming a successful write is on disk.
fsync and durability	Applications that must not lose data call fsync (or open files with O_SYNC) so the kernel writes the file to stable storage before returning.	fsync and durability Applications that must not lose data call fsync (or open files with O_SYNC) so the kernel writes the file to stable...
Finding a slow disk	Columns to watch: await (average time per request including queueing, in ms), aqu-sz (queue length) and %util (how busy the device is; on SSDs and arrays near 100...	Using %util alone to judge SSDs.
Limits that surprise people	Inodes: df -i shows inode use; millions of tiny files can exhaust inodes while df -h shows free space.	Trying to kill processes stuck on a dead network mount.

Flow: The path of a read and a write to fsync and durability / fsync and durability to Finding a slow disk / Finding a slow disk to Limits that surprise people

Incident: The log disk that went read-only

Incident: *An application started failing with errors writing to its log directory, although df showed free space.*

```
$ touch /var/log/app/test
touch: cannot touch '/var/log/app/test': Read-only file system
$ dmesg -T | tail -3
[Wed Jan  7 03:11:09 2026] EXT4-fs error (device sdb1): ext4_find_entry: reading directory lblock  \
    0
[Wed Jan  7 03:11:09 2026] EXT4-fs (sdb1): Remounting filesystem read-only
```

The kernel remounted the filesystem read-only after I/O errors from the failing disk, to protect data. The team moved the service to a healthy volume, replaced the disk, ran fsck on the unmounted filesystem, and added an alert on disk errors in the kernel log.

Common mistakes

- Assuming a successful write is on disk
- Using %util alone to judge SSDs
- Ignoring inode exhaustion
- Trying to kill processes stuck on a dead network mount

Check yourself

What does a write usually do before returning?

Answer: Puts data in the page cache as dirty pages

What makes sure data reaches stable storage?

Answer: fsync

Which iostat column shows average request latency?

Answer: await

MARGA by EknathaLabs marga.eknathalabs.com

df -h shows free space but file creation fails. What else do you check?

Answer: Inode usage with df -i

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 21. Networking in the kernel: sockets, queues and conntrack

14 minutes. Understand sockets, listen queues, TCP states and connection tracking, and use ss and conntrack to find connection problems.

When a service drops connections, applications often see only timeouts. The truth is in the kernel: full accept queues, sockets stuck in TIME_WAIT, exhausted ports, or a connection table that is full. A small set of commands shows these structures directly, and a few kernel parameters control them.

What you will learn

- Sockets, ports and the listen and accept queues
- Read ss output and TCP states
- Port exhaustion, TIME_WAIT and keepalives
- Netfilter and conntrack, and what happens when the table fills

Sockets and queues

A **socket** is the kernel object behind a connection. A server calls `listen`, and incoming connections wait in a queue until the application calls `accept`. If the application is slow, the queue fills (its size is limited by the backlog argument and `net.core.somaxconn`), and new connections are dropped or delayed, so clients see timeouts although the server appears fine.

```
ss -ltn
State   Recv-Q  Send-Q  Local Address:Port
LISTEN  129     128     0.0.0.0:8080
```

For listening sockets, `Recv-Q` is the current accept queue length and `Send-Q` its maximum. Here 129 is above 128: the queue is overflowing.

TCP states with ss

```
ss -s
ss -tan state established | wc -l
ss -tan state time-wait | wc -l
ss -tanp | grep :5432 | head
ss -ti dst 10.0.1.20
```

`ss` replaces `netstat`. States to know: `ESTABLISHED` (active), `SYN-SENT` (connecting, no answer yet), `SYN-RECV`, `TIME-WAIT` (closed connection kept briefly to catch late packets; large numbers are normal on busy clients) and `CLOSE-WAIT` (the remote side closed but **your application has not closed the socket**: a bug when it piles up). `-p` shows the process, `-i` shows TCP internals such as retransmissions and round-trip time.

Ports and timeouts

A client uses one local port per connection to the same destination. The range comes from `net.ipv4.ip_local_port_range` (often about 28,000 ports). A client making many short connections to one server can run out, giving `Cannot assign requested address`. The cure is connection reuse (keep-alive, pooling), not simply a bigger range. Keepalive settings (`net.ipv4.tcp_keepalive_time`) control how long idle connections are probed; load balancers and NAT devices drop idle connections silently, so applications should use their own keepalive or shorter timeouts.

Netfilter and conntrack

Firewall rules and NAT (iptables, nftables, Kubernetes service routing) rely on **connection tracking**: the kernel remembers every flow in a table.

```
sudo conntrack -L | head
sudo conntrack -C
sysctl net.netfilter.nf_conntrack_max
dmesg -T | grep conntrack
```

MARGA by EknathaLabs marga.eknathalabs.com

If `conntrack -C` approaches `nf_conntrack_max`, new connections are dropped with `table full` in the kernel log. Fixes: raise the limit (it costs memory), shorten timeouts for UDP and closed flows, avoid tracking traffic that does not need it, and reduce connection churn. On busy NAT gateways and Kubernetes nodes this is a classic cause of mysterious packet loss.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Networking in the kernel: sockets, queues and... fits together

Part	What it does	What goes wrong
Sockets and queues	A socket is the kernel object behind a connection.	Blaming the network for application accept delays.
TCP states with ss	ss replaces netstat. States to know: ESTABLISHED (active), SYN-SENT (connecting, no answer yet), SYN-RECV, TIME-WAIT (closed connection kept briefly to catch late...	Treating TIME_WAIT as a bug.
Ports and timeouts	A client uses one local port per connection to the same destination.	A client making many short connections to one server can run out, giving Cannot assign requested address.
Netfilter and conntrack	Firewall rules and NAT (iptables, nftables, Kubernetes service routing) rely on connection tracking: the kernel remembers every flow in a table.	Raising conntrack limits without checking memory.

Flow: Sockets and queues to TCP states with ss / TCP states with ss to Ports and timeouts / Ports and timeouts to Netfilter and conntrack

Incident: Thousands of CLOSE_WAIT

Incident: *A service gradually stopped accepting new work and file descriptors ran out.*

```
$ ss -tan state close-wait | wc -l
28450
$ ls /proc/$(pidof orders)/fd | wc -l
65530
$ ss -tanp state close-wait | head -2
CLOSE-WAIT 1 0 10.0.2.15:41222 10.0.9.3:5432 users:(("orders",pid=4121,fd=311))
```

The application did not close database connections after errors, so the sockets stayed in CLOSE_WAIT until the process ran out of file descriptors. The fix closed connections in a finally block, used a pool with limits and idle timeouts, and added alerts on CLOSE_WAIT count and open descriptors.

Common mistakes

- Blaming the network for application accept delays
- Treating TIME_WAIT as a bug
- Ignoring CLOSE_WAIT build-up
- Raising conntrack limits without checking memory

Check yourself

What does a growing number of CLOSE_WAIT sockets suggest?

Answer: The application is not closing sockets after the peer closed them

What does Recv-Q mean for a LISTEN socket in ss?

Answer: Connections waiting in the accept queue

Which kernel log message shows connection tracking is overloaded?

Answer: nf_conntrack: table full, dropping packet

MARGA by EknathaLabs marga.eknathalabs.com

How do you avoid ephemeral port exhaustion?

Answer: Reuse connections with keep-alive or pooling

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 22. Debugging a sick Linux server

16 minutes. Use a fixed order of checks (CPU, memory, disk, network, logs) to find why a server is slow or failing, and know which command answers which question.

A server is "slow". That word hides a dozen different problems: a runaway process, memory pressure, a full disk, a saturated network link, or a dependency that stopped answering. Guessing wastes time. Experienced engineers walk the same short checklist every time, so that the machine itself tells them where the trouble is.

What you will learn

- A repeatable order of checks for a sick server
- The one command that answers each question
- How to read load, memory, disk and network numbers without being fooled
- How to attach to a misbehaving process and see what it is doing

Start with the questions, not the tools

Ask four things, in this order: what changed recently, what do the logs say, which resource is saturated, and which process is responsible. The **USE method** helps with the third: for every resource, check Utilisation, Saturation and Errors.

Resource	Quick check	Warning sign
CPU	`uptime`, `top`	Load average far above the number of cores
Memory	`free -m`, `dmesg -T`	Very little available memory, swap in use, OOM kills
Disk space	`df -h` and `df -i`	100 percent used, or no free inodes
Disk speed	`iostat -x 1 3`	High `%util` and long `await` times
Network	`ss -s`, `ip -s link`	Many connections in odd states, errors or drops

The first two minutes

```
$ uptime
 14:03:11 up 41 days,  3:12,  2 users,  load average: 9.80, 8.40, 5.10
$ nproc
4
$ top -b -n1 | head -12
$ free -m
              total        used        free      shared  buff/cache   available
Mem:           7822        7410         112          12         300          95
$ df -h /var; df -i /var
```

A load of 9.8 on 4 cores means work is queueing. Load counts processes that want CPU and also those blocked on disk, so look at `top` to see which it is: high `%us`/`%sy` means CPU; high `%wa` means waiting for disk. In `free -m`, trust the **available** column, not "free", because the kernel uses spare memory as cache.

Find the process and see what it does

```
ps aux --sort=-%mem | head -5          # biggest memory users
ss -tnp state established | head       # who is connected, and which process owns it
lsof -p 4321 | head -20                # files and sockets a process has open
sudo strace -p 4321 -f -tt -e trace=network,file   # live system calls (use briefly, it slows the  \
    process)
journalctl -u shop --since "-15min"    # what the service itself reported
```

`strace` shows what a process is asking the kernel to do. If it repeats the same failing call, you have found the loop. Remove it as soon as you have the answer.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: A fixed order for a sick server

Part	What it does	What goes wrong
What changed?	Deploys, config edits, package updates and traffic changes in the last hours.	Skipping this step means hours spent on a cause that was introduced this morning.
Logs and kernel messages	journalctl and dmesg show crashes, OOM kills and disk errors with timestamps.	Logs rotated away or never kept leave you guessing.
CPU, memory, disk, network	uptime, free, df, iostat and ss find the saturated resource.	Reading the wrong column, such as free instead of available memory, misleads you.
Responsible process	ps, top, lsof and strace link the saturation to one program.	Killing the wrong process fixes nothing and may hide the evidence.
Fix and verify	Change one thing, then confirm the metric returns to normal.	Several changes at once leave you unsure what worked.

Flow: What changed? to Logs and kernel messages / What changed? to CPU, memory, disk, network / Logs and kernel messages to Responsible process / CPU, memory, disk, network to Responsible process / Responsible process to Fix and verify

Incident: the server with free space and no space

Incident: *Uploads fail at 09:40 with "No space left on device". Disk graphs show 60 percent free.*

```
$ df -h /srv
/dev/sdb1   200G  120G   80G  60% /srv
$ df -i /srv
Filesystem   Inodes   IUsed IFree IUse% Mounted on
/dev/sdb1   13107200 13107200    0  100% /srv
$ sudo find /srv/sessions -type f | wc -l
12984311
```

The session directory held almost 13 million tiny files, and a cleanup job had been failing for weeks. The team removed files older than a day, fixed the job, added an alert on inode use, and moved sessions to a store designed for them.

Common mistakes

- Restarting the service before collecting evidence (logs, a thread dump, `ss` output)
- Reading "free" memory instead of "available"
- Changing several settings at once
- Leaving `strace` or debug logging on after the investigation

Check yourself

A server shows a load average of 12 on 4 cores, but CPU use is only 10 percent and the %wa value is high. What is the likely cause?

Answer: Processes are waiting on disk or other I/O

Which memory number should you trust to see how much can be used without swapping?

Answer: available

Which command shows the system calls a running process makes?

Answer: strace

A disk is 60 percent free, but writes fail with "No space left on device". What do you check?

Answer: Inode usage with df -i

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 23. Troubleshooting playbook: method and the first five minutes

12 minutes. Triage a slow or failing Linux server in order: ask what changed, check each resource for use, saturation and errors, and narrow the cause before you touch anything.

A page says "the server is slow". Most people open `top`, see something odd and start changing settings. A playbook replaces that habit with a fixed order of questions, so you find the cause faster and do not make things worse. This lesson is the front door to the playbook lessons that follow.

What you will learn

- Ask the questions that narrow the problem before you run commands
- Apply the USE method: utilization, saturation and errors for each resource
- Run a first-five-minutes command set and read the results
- Record what you saw so a handover or post-mortem is easy

Start with questions, not commands

Before the shell, spend one minute on: **What is the symptom, exactly?** (errors, latency, a crash). **Since when? What changed** (a deploy, a config change, a certificate, a traffic spike, a kernel update)? **Who is affected** (one host, one region, everyone)? **Is it getting worse?** A recent change is the cause far more often than a random failure, and "only one host" points to the host, while "everyone" points to a shared dependency.

The USE method

For every resource (CPU, memory, disks, network, and also queues and locks) check three things:

- **Utilization**: how busy is it, as a percentage of time or capacity
- **Saturation**: is work waiting, such as a run queue, swap activity or an I/O queue
- **Errors**: counts of faults, such as disk errors, packet drops or OOM kills

Saturation is the signal people forget. A CPU at 100 percent with no queue is working hard but healthy. A disk at 60 percent with a long queue is already hurting users.

```
$ uptime
$ dmesg -T | tail -20
$ vmstat 1 5
$ mpstat -P ALL 1 3
$ pidstat 1 3
$ iostat -xz 1 3
$ free -m
$ ss -s
```

Read them in this order: `uptime` gives load trend, `dmesg` shows kernel errors (OOM kills, disk and network faults), `vmstat` shows run queue (`r`), blocked (`b`), swap (`si`, `so`) and CPU split, `mpstat` shows whether one core is stuck, `pidstat` names the busy process, `iostat` shows disk `await` and `%util`, `free` shows available memory, and `ss -s` summarizes sockets.

Narrow and then confirm

Pick the resource with the clearest saturation or error, find the process or workload behind it, and form one hypothesis. Test it with a read-only command first. Change one thing at a time, and write down the time and the change. If a fix does not help, undo it, so that you never stack unknown changes.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Troubleshooting playbook: method and the first... fits together

Part	What it does	What goes wrong
Start with questions, not commands	Before the shell, spend one minute on: What is the symptom, exactly?	Reading only utilization and ignoring queues and errors.
The USE method	For every resource (CPU, memory, disks, network, and also queues and locks) check three things.	Skipping dmesg, where the kernel often already named the problem.
Narrow and then confirm	Pick the resource with the clearest saturation or error, find the process or workload behind it, and form one hypothesis.	Changing settings before you know the resource that is saturated.

Flow: Start with questions, not commands to The USE method / The USE method to Narrow and then confirm

Incident: The nightly slowdown

Incident: *Every night at 02:10 an API on one server became slow for about 15 minutes, and the team blamed the new release.*

```
$ iostat -xz 1 3
Device  r/s   w/s   await  %util
sda     12    480   310.2  99.8
$ pidstat -d 1 3 | sort -k5 -nr | head -3
```

The release was innocent. A log compression job started by cron at 02:10 wrote heavily to the same disk, and disk wait time rose to 310 ms. The fix was to move the job to a separate volume and run it with `ionice -c3`. The team also added disk wait to the dashboard so the next slowdown would show the real resource.

Common mistakes

- Changing settings before you know the resource that is saturated
- Reading only utilization and ignoring queues and errors
- Skipping `dmesg`, where the kernel often already named the problem
- Making several changes at once and losing track of what helped

Check yourself

What does the USE method check for each resource?

Answer: Utilization, saturation and errors

Why ask "what changed" first?

Answer: Recent changes cause most incidents, so it narrows the search quickly

A disk is 60 percent utilized with a long request queue. How healthy is it?

Answer: Not healthy: queued requests mean saturation that users feel

A fix does not help. What should you do?

Answer: Undo it, then test the next hypothesis

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 24. Troubleshooting playbook: high load and CPU saturation

12 minutes. Tell CPU saturation from I/O waits and steal time, find the busy process or thread, and decide whether to fix, limit or scale.

"The load is 30" can mean three very different problems: the CPUs are busy, processes are stuck waiting on disk or network storage, or a noisy neighbour is taking CPU from a virtual machine. Each has a different cure, so the first job is to tell them apart.

What you will learn

- Read load average correctly, including tasks in uninterruptible sleep
- Split CPU time into user, system, iowait and steal
- Find the busy process, thread or kernel path
- Choose between fixing, limiting and scaling

What load average counts

Load average is the average number of tasks that are runnable **or** in uninterruptible sleep (state `D`, usually waiting for I/O) over 1, 5 and 15 minutes. Compare it with the CPU count from `nproc`. A load of 16 on 16 cores is a full machine; a load of 16 on 2 cores is a queue. A high load with low CPU use points to blocked tasks, not computation.

```
$ nproc
$ uptime
$ mpstat -P ALL 1 3
$ ps -eo state,pid,cmd | awk '$1 ~ /^D/' | head
```

Read the CPU split

In `mpstat` or `top`:

- `%usr` high: application code is computing. Profile it.
- `%sys` high: time in the kernel, often many system calls, context switches, network packets or locks.
- `%iowait` high: CPUs are idle because tasks wait for storage. Look at disks and network filesystems, not at CPU.
- `%steal` high: the hypervisor is giving your virtual CPU to someone else. Ask for more capacity or move the VM.
- One core at 100 percent while others idle: a single-threaded task is the limit, and adding cores will not help.

Find the culprit

```
$ top -H -o %CPU
$ pidstat -u -t 1 3
$ sudo perf top -p 2314
$ cat /proc/2314/status | grep -i ctxt
```

`top -H` shows threads, which matters for runtimes with thread pools. `pidstat -t` lists per-thread CPU. `perf top` shows which functions burn CPU. A very high count of voluntary context switches suggests lock waits or I/O; non-voluntary switches suggest CPU contention.

Decide: fix, limit or scale

If one process is wrong (an infinite loop, a bad query, a retry storm), fix or restart it and keep evidence. If a legitimate job competes with a service, limit it with `nice`, `ionice` or a systemd `CPUQuota=`. If demand is real and steady, scale out or up. Scale only after you know the bottleneck: more cores do nothing for an I/O wait.

MARGA by EknathaLabs
marga.eknathalabs.com

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Troubleshooting playbook: high load and CPU... fits together

Part	What it does	What goes wrong
What load average counts	Load average is the average number of tasks that are runnable or in uninterruptible sleep (state D, usually waiting for I/O) over 1, 5 and 15 minutes.	Reading load average without comparing it with the core count.
Read the CPU split	In mpstat or top.	Adding CPUs for a problem that is I/O wait.
Find the culprit	top -H shows threads, which matters for runtimes with thread pools.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Decide: fix, limit or scale	If one process is wrong (an infinite loop, a bad query, a retry storm), fix or restart it and keep evidence.	Restarting the service before capturing stack or profile evidence.

Flow: What load average counts to Read the CPU split / Read the CPU split to Find the culprit / Find the culprit to Decide: fix, limit or scale

Incident: Load 80 and an idle CPU

Incident: *A build server showed load 80, yet* `top` *reported 95 percent idle, and builds hung.*

```
$ ps -eo state,pid,cmd | awk '$1 ~ /^D/' | head -3
D 4410 /usr/bin/git-archive-helper
$ mount | grep nfs
10.0.8.5:/builds on /mnt/builds type nfs
$ dmesg -T | grep -i "not responding"
nfs: server 10.0.8.5 not responding, still trying
```

Dozens of processes were in `D` state, waiting for an NFS server that had stopped answering. The CPU was idle because nothing could run. The fix was on the storage side, plus soft mount options with timeouts for the build share and an alert on NFS "not responding" messages. Killing processes would not have helped, because tasks in `D` state cannot be killed until the I/O returns.

Common mistakes

- Adding CPUs for a problem that is I/O wait
- Reading load average without comparing it with the core count
- Ignoring steal time on virtual machines
- Restarting the service before capturing stack or profile evidence

Check yourself

Which tasks does load average count?

Answer: Runnable tasks and tasks in uninterruptible sleep

%steal is high on a virtual machine. What does it mean?

Answer: The hypervisor is giving your CPU time to other guests

One core is at 100 percent and the rest are idle. What is likely?

Answer: A single-threaded task is the bottleneck

Why can you not kill a process in D state?

Answer: It waits inside the kernel for I/O and cannot handle signals until it returns

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 25. Troubleshooting playbook: memory pressure, swap and the OOM killer

12 minutes. Read free, meminfo and kernel logs correctly, spot swap thrashing and leaks, and tell a host-level OOM kill from a container memory limit.

Linux uses spare memory for caches, so "free memory is low" is usually normal. The real questions are whether applications can still get memory, whether the system is swapping heavily, and whether the kernel has killed something. This lesson gives you a way to answer each.

What you will learn

- Read `free` and `/proc/meminfo` without panic over the page cache
- Spot swap thrashing in `vmstat`
- Find OOM kills in the kernel log and understand why a victim was chosen
- Tell a host OOM from a cgroup or container limit kill

Available, not free

```
$ free -m
$ grep -E 'MemAvailable|SwapTotal|SwapFree|Dirty' /proc/meminfo
$ ps -eo pid,rss,cmd --sort=-rss | head -5
```

The column that matters is **available**: memory that can be given to programs without swapping, including reclaimable cache. Low "free" with healthy "available" is fine. Sort by resident set size (`rss`) to find the large processes, and watch whether one grows over time, which suggests a leak.

Swap and thrashing

A little swap use can be fine, since idle pages move out. The danger is **thrashing**: the system continuously swaps pages in and out, and everything slows.

```
$ vmstat 1 5
procs -----memory---- ---swap-- -----io---- -system-- ------cpu-----
 r  b   swpd   free   si   so    bi    bo   in   cs us sy id wa st
 3  6 4194304  81200  912  1604  8800  9100  900  1500  4  6 20 70  0
```

High `si` and `so` together with high `wa` show thrashing. The cure is to free memory (stop or restart the biggest consumer, reduce limits), not to add more swap. Setting `vm.swappiness` lower makes the kernel prefer dropping cache over swapping, which can help latency-sensitive servers.

The OOM killer

When memory cannot be reclaimed, the kernel's OOM killer picks a process with a high `oom_score` and ends it.

```
$ dmesg -T | grep -i -E 'out of memory|killed process'
$ journalctl -k --since "1 hour ago" | grep -i oom
$ cat /proc/2314/oom_score
```

The log shows who was killed, how much memory it used and the state of memory at that time. The victim is often the largest process, not necessarily the one that leaked. You can protect critical processes with `oom_score_adj` (a negative value) in a systemd unit using `OOMScoreAdjust=`, but protecting one process shifts the kill to another, so fix the cause as well.

Host or container?

Containers run in cgroups with their own limit. When a container exceeds `memory.max`, the kernel kills a process inside that cgroup, even if the host has plenty of memory free. In Kubernetes this shows as `OOMKilled` with exit code 137. Check the limit, the working set and whether the runtime (for example a JVM heap setting) is sized against the limit, not against the host.

MARGA by EknathaLabs
marga.eknathalabs.com

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Troubleshooting playbook: memory pressure, swap... fits together

Part	What it does	What goes wrong
Available, not free	The column that matters is available: memory that can be given to programs without swapping, including reclaimable cache.	Treating low "free" memory as a problem.
Swap and thrashing	A little swap use can be fine, since idle pages move out.	Adding swap to hide a leak.
The OOM killer	When memory cannot be reclaimed, the kernel's OOM killer picks a process with a high oom_score and ends it.	The OOM killer When memory cannot be reclaimed, the kernel's OOM killer picks a process with a high oom_score and ends it.
Host or container?	Containers run in cgroups with their own limit.	Sizing runtime heaps against the host, not the container limit.

Flow: Available, not free to Swap and thrashing / Swap and thrashing to The OOM killer / The OOM killer to Host or container?

Incident: The service that died at night

> **Incident:** *A Java service restarted every few hours with no error in its own log, and the host showed plenty of free memory.*

```
$ dmesg -T | grep -i killed
Memory cgroup out of memory: Killed process 5120 (java) total-vm:9400000kB, anon-rss:2095000kB
$ cat /sys/fs/cgroup/system.slice/app.service/memory.max
2147483648
```

The service ran in a systemd cgroup with a 2 GiB limit, but the JVM was configured with a 3 GiB heap. The kernel killed it as soon as the heap grew past the limit, and the application never had a chance to log. The team set the heap to about 70 percent of the limit, added a memory alert on the working set and kept the kernel log in their incident checklist.

Common mistakes

- Treating low "free" memory as a problem
- Adding swap to hide a leak
- Sizing runtime heaps against the host, not the container limit
- Looking only at application logs, which never see a SIGKILL

Check yourself

Which memory figure shows what programs can still get?

Answer: Available

What indicates swap thrashing?

Answer: High si and so together with high iowait

A container is killed with exit code 137 although the host has memory. Likely cause?

Answer: It exceeded its cgroup memory limit

Where do you find proof that the OOM killer ran?

Answer: The kernel log, through dmesg or journalctl -k

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 26. Troubleshooting playbook: disk full, inodes and I/O stalls

12 minutes. Find what fills a filesystem, including inodes and deleted-but-open files, and read iostat to separate a busy disk from a slow one.

A full disk stops databases, breaks logins and silently corrupts nothing but still takes services down. Slow disks cause the opposite symptom: everything works but crawls. Both have short checklists.

What you will learn

- Find the space hog with `df` and `du`
- Recognize inode exhaustion and deleted files still held open
- Read `iostat` columns: `await`, queue size and `%util`
- Limit the damage of noisy jobs with `ionice` and quotas

Where did the space go?

```
$ df -hT
$ df -i
$ sudo du -xh --max-depth=1 /var | sort -h | tail
$ sudo ncdu -x /var
```

Use `df -h` for space and `df -i` **for inodes**. A filesystem can have free blocks but no free inodes when it holds millions of tiny files (mail queues, session files, cache directories). The symptom is `No space left on device` while `df -h` shows space available. The `-x` flag keeps `du` on one filesystem so it does not wander into other mounts.

Space that du cannot see

If `df` says 100 percent but `du` finds far less, a process may still hold a **deleted file open**. The name is gone, but the blocks stay allocated until the process closes the file.

```
$ sudo lsof +L1 | head
$ sudo ls -l /proc/2210/fd | grep deleted
$ sudo truncate -s 0 /proc/2210/fd/7
```

Restarting the process releases the space. Truncating through `/proc/PID/fd/N` frees it without a restart. The root fix is proper log rotation that signals or reopens the file (`copytruncate` or a reload hook).

Slow disks

```
$ iostat -xz 1 3
Device  r/s  w/s  rkB/s  wkB/s  aqu-sz  await  %util
nvme0n1 120  900  4800   52000  18.4    22.1   98.0
```

`await` is the average time a request waits and is served, in milliseconds. `aqu-sz` is the average queue length. `%util` near 100 on a single spinning disk means saturated, but on modern NVMe or arrays it can reach 100 while still serving more, so judge by `await` and queue length against what is normal for that device. Use `pidstat -d` or `iotop -o` to see which process does the I/O.

Protect production from heavy jobs

Run backups, compression and index builds with `ionice -c3 nice -n19 command` so they use only idle I/O. Put logs and data on separate filesystems, set quotas for user and cache directories, and alert when a filesystem passes 80 percent or when its growth rate predicts it will fill within a day.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Troubleshooting playbook: disk full, inodes and... fits together

Part	What it does	What goes wrong
Where did the space go?	Use df -h for space and df -i for inodes.	Forgetting inodes when space looks free.
Space that du cannot see	If df says 100 percent but du finds far less, a process may still hold a deleted file open.	Deleting a log that a process still holds open.
Slow disks	await is the average time a request waits and is served, in milliseconds.	Judging an SSD only by %util instead of await and queue depth.
Protect production from heavy jobs	Run backups, compression and index builds with ionice -c3 nice -n19 command so they use only idle I/O.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Where did the space go? to Space that du cannot see / Space that du cannot see to Slow disks / Slow disks to Protect production from heavy jobs

Incident: Full disk with nothing in it

> **Incident:** *A database host reported "no space left on device", yet the data directory was only 40 percent of the disk.*

```
$ df -h /var
/dev/sdb1  200G  200G  0  100% /var
$ sudo lsof +L1 | head -3
java  2210  app  7w  REG  8,17  96000000000  0  /var/log/app/debug.log (deleted)
```

Someone had deleted a 90 GB debug log to free space, but the application still had it open and kept writing. The space was not released until the process was restarted. The team restarted it in a quiet window, set log rotation with a size limit and a reload hook, and added a filesystem alert at 80 percent.

Common mistakes

- Running `du` across mounts and chasing the wrong directory
- Forgetting inodes when space looks free
- Deleting a log that a process still holds open
- Judging an SSD only by `%util` instead of `await` and queue depth

Check yourself

A write fails with "No space left on device", but df -h shows free space. What next?

Answer: Check df -i for inode exhaustion

How do you find deleted files that are still held open?

Answer: lsof +L1

Which iostat column shows how long requests wait and are served?

Answer: await

How should a heavy backup job run on a busy server?

Answer: With ionice -c3 and a low nice value so it uses idle I/O

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 27. Troubleshooting playbook: network drops and slow connections on a host

13 minutes. Check a Linux host's network path in layers: interface errors, MTU, retransmits, DNS delay and connection tracking, then capture packets only when you need proof.

"The network is slow" is a claim, not a diagnosis. On a single host you can check each layer in a few minutes and either find the fault or hand the network team evidence that it is not on the host.

What you will learn

- Check interface counters, link speed and drops
- Spot retransmits and MTU problems
- Separate DNS delay from connection delay
- Take a focused packet capture

Layer by layer

Work from the bottom up. First the link and the interface:

```
$ ip -s link show eth0
$ ethtool eth0 | grep -E 'Speed|Duplex|Link detected'
$ ethtool -S eth0 | grep -i -E 'err|drop|miss'
$ ip route get 10.0.8.5
```

Growing `errors`, `dropped` or `overruns` counters point to a cable, driver, ring buffer or overloaded interface. A link negotiated at 100 Mb/s instead of 10 Gb/s is a classic hidden fault. `ip route get` shows which interface and gateway the kernel will really use.

Retransmits and MTU

```
$ nstat -az | grep -i -E 'retrans|listendrops'
$ ss -ti dst 10.0.8.5 | head
$ ping -M do -s 1472 -c 3 10.0.8.5
```

A rising retransmit count means packets are lost somewhere on the path. The `ping -M do -s 1472` test forbids fragmentation: if it fails while small pings work, the path MTU is smaller than expected (tunnels, VPNs and overlay networks often reduce it), so large responses hang while small ones succeed. The fix is to set the right MTU or clamp the TCP MSS, not to retry.

Is it DNS?

Measure the phases separately. `curl` can print timings:

```
$ curl -s -o /dev/null -w 'dns=%{time_namelookup} connect=%{time_connect} tls=%{time_appconnect} t \
    otal=%{time_total}\n' https://api.example.com/
dns=5.003 connect=5.021 tls=5.080 total=5.150
```

Here five seconds went to the name lookup, and the rest was fast. Check `/etc/resolv.conf` (unreachable first nameserver, long search list, timeouts), and compare `dig` against each resolver. A fixed 5-second delay is a typical resolver timeout followed by a retry.

Connection tracking and capture

Run `conntrack -C` against `nf_conntrack_max`, and check `dmesg` for `nf_conntrack: table full`. When you need proof, capture a narrow slice:

```
$ sudo tcpdump -i eth0 -nn -c 200 host 10.0.8.5 and port 443 -w /tmp/case.pcap
```

Filter by host and port, limit the count, and write to a file for analysis in Wireshark. Keep captures short and delete them afterwards, because they can contain sensitive data.

MARGA by EknathaLabs marga.eknathalabs.com

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Troubleshooting playbook: network drops and slow... fits together

Part	What it does	What goes wrong
Layer by layer	Work from the bottom up.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Retransmits and MTU	A rising retransmit count means packets are lost somewhere on the path.	Ignoring MTU on tunnels and overlays.
Is it DNS?	Measure the phases separately.	Blaming the network without measuring DNS, connect and TLS time.
Connection tracking and capture	Run conntrack -C against nf_conntrack_max, and check dmesg for nf_conntrack: table full.	Leaving packet captures with sensitive data lying around.

Flow: Layer by layer to Retransmits and MTU / Retransmits and MTU to Is it DNS? / Is it DNS? to Connection tracking and capture

Incident: Five seconds, every time

Incident: *A service made calls to an internal API that took exactly five seconds now and then, while the API itself answered in 20 ms.*

```
$ curl -s -o /dev/null -w 'dns=%{time_namelookup} total=%{time_total}\n' http://billing.internal/h \
    ealth
dns=5.002 total=5.030
$ cat /etc/resolv.conf
nameserver 10.0.0.2
nameserver 10.0.0.3
```

The first nameserver had been decommissioned. Every lookup waited five seconds for it to time out, then used the second one. The team removed the dead address, added the resolver health to their checks and set `options timeout:1 attempts:2` as a safety net. No network change was needed.

Common mistakes

- Blaming the network without measuring DNS, connect and TLS time
- Ignoring MTU on tunnels and overlays
- Capturing everything and filling the disk
- Leaving packet captures with sensitive data lying around

Check yourself

What does a rising retransmit counter suggest?

Answer: Packets are being lost on the path

Which test finds a path MTU problem?

Answer: ping with the do-not-fragment flag and a large size

A request shows dns=5.0 seconds and everything else fast. Where is the fault?

Answer: In name resolution, such as a dead nameserver

How should you take a packet capture?

Answer: Narrow by host and port, limit the size, and delete it afterwards

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 28. Troubleshooting playbook: boot failures and services that will not start

12 minutes. Diagnose failed units with systemctl and journalctl, find slow or broken boot steps, and recover from emergency mode caused by mounts and configuration mistakes.

A reboot is the moment hidden mistakes appear: an unmounted disk, a config file with a typo, a service that depended on something that no longer starts first. Linux gives clear evidence if you know where to look.

What you will learn

- Read `systemctl status` and `journalctl` for a failing unit
- Use `systemd-analyze` to find slow or blocked boot steps
- Recover from emergency mode caused by `fstab` errors
- Prevent repeat failures with ordering, timeouts and `nofail`

A service will not start

```
$ systemctl status payments.service
$ journalctl -u payments.service -b --no-pager | tail -40
$ systemctl cat payments.service
$ systemctl list-dependencies payments.service
$ systemd-analyze verify /etc/systemd/system/payments.service
```

`status` shows the state, the last exit code and recent log lines. `journalctl -u UNIT -b` gives the full log for this boot. `systemctl cat` shows the unit with all drop-ins, which is where surprising overrides hide. Typical causes: a missing file or permission, a port already in use, a wrong user, an environment variable that is not set, and a dependency that is not ready (`After=` and `Requires=` describe order and need separately).

What slowed the boot

```
$ systemd-analyze
$ systemd-analyze blame | head
$ systemd-analyze critical-chain
```

`blame` lists units by the time they took, and `critical-chain` shows the sequence that actually decided boot time. A unit waiting for a network mount or a DNS name is a common culprit, and the fix is a sensible timeout or a dependency on `network-online.target` only where it is really needed.

Emergency mode and fstab

If a filesystem in `/etc/fstab` cannot be mounted, systemd may drop to emergency mode and wait at a prompt. Read the error, then fix the line:

```
$ journalctl -xb -p err | head -20
$ findmnt --verify
$ blkid
$ systemctl daemon-reload
```

Use UUIDs from `blkid` instead of device names, which can change. For non-critical mounts add `nofail` and `x-systemd.device-timeout=10s`, so a missing disk does not block the whole system. For a broken kernel or boot loader, boot an older kernel entry from the boot menu, or use a rescue image, mount the root filesystem and repair it. On cloud instances, attach the disk to a helper instance when the console is not available.

Before you reboot

Validate changes before the reboot that will test them: `findmnt --verify` for mounts, `systemd-analyze verify` for units, `sshd -t` for SSH config, `nginx -t` for web configs. Keep a working console or out-of-band access open, and make sure a rollback exists.

MARGA by EknathaLabs marga.eknathalabs.com

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Troubleshooting playbook: boot failures and... fits together

Part	What it does	What goes wrong
A service will not start	status shows the state, the last exit code and recent log lines.	Reading only status and never the full journal for that boot.
What slowed the boot	blame lists units by the time they took, and critical-chain shows the sequence that actually decided boot time.	Making every mount essential for boot.
Emergency mode and fstab	If a filesystem in /etc/fstab cannot be mounted, systemd may drop to emergency mode and wait at a prompt.	Using device names like /dev/sdc1 in fstab.
Before you reboot	Validate changes before the reboot that will test them: findmnt --verify for mounts, systemd-analyze verify for units, sshd -t for SSH config, nginx -t for web configs.	Rebooting with unvalidated changes and no console access.

Flow: A service will not start to What slowed the boot / What slowed the boot to Emergency mode and fstab / Emergency mode and fstab to Before you reboot

Incident: The extra disk that stopped the boot

Incident: *After a routine kernel patch and reboot, a server did not come back on the network.*

```
$ journalctl -xb -p err | head -5
Dependency failed for /data.
You are in emergency mode.
$ grep data /etc/fstab
/dev/sdc1  /data  ext4  defaults  0  2
```

A data disk had been detached weeks earlier, and its device name had been reused by a different disk. The mount failed and systemd chose emergency mode, so SSH never started. After console access, the team replaced the line with a UUID-based entry with `nofail`, and added a pre-reboot check of `findmnt --verify` to the patching runbook.

Common mistakes

- Rebooting with unvalidated changes and no console access
- Using device names like `/dev/sdc1` in fstab
- Making every mount essential for boot
- Reading only `status` and never the full journal for that boot

Check yourself

Which command shows the full log of a unit for this boot?

Answer: journalctl -u UNIT -b

What does the nofail option do in fstab?

Answer: Boot continues even if that filesystem fails to mount

How do you see which units made boot slow?

Answer: systemd-analyze blame and critical-chain

What should you do before a risky reboot?

Answer: Validate mounts and configs and keep console access ready

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 29. Debugging with strace: watch system calls

13 minutes. Use strace to see which files, sockets and system calls a stuck or failing program uses, and read the output to find the real cause.

Every program talks to Linux through system calls: open a file, connect to a socket, read, write, wait. strace prints those calls as they happen, so it can explain failures that logs never mention. It is often the fastest way to learn what a program is really doing.

What you will learn

- What a system call is and how strace shows it
- Reading open, connect and read calls and their error codes
- Following child processes and timing calls
- Attaching to a running process and using strace safely

What strace shows

A program cannot touch files, the network or memory directly; it asks the kernel through **system calls**. `strace` starts a program (or attaches to one) and prints each call, its arguments and the result.

```
$ strace -f -tt -e trace=file,network ./myapp 2>&1 | head
12:01:03.114 openat(AT_FDCWD, "/etc/app/config.yml", O_RDONLY) = -1 ENOENT (No such file or direct \
    ory)
12:01:03.115 openat(AT_FDCWD, "/usr/local/etc/app.yml", O_RDONLY) = -1 ENOENT (No such file or dir \
    ectory)
12:01:03.116 connect(3, {sa_family=AF_INET, sin_port=htons(5432), sin_addr=inet_addr("10.0.0.5")}, \
     16) = -1 ECONNREFUSED
```

A result of `-1` followed by an error name is a failure. The name is the clue.

Error	Meaning	Usual cause
ENOENT	No such file or directory	Wrong path or missing file
EACCES	Permission denied	Owner, mode or security profile
ECONNREFUSED	Nothing listens on that port	Service down or wrong port
ETIMEDOUT	No answer	Firewall or routing
EADDRINUSE	Port already used	Another process holds it

Diagram: How strace sits between a program and the kernel

Part	What it does	What goes wrong
Your program	Runs normally but each system call is intercepted.	A crash inside the program may show only a signal at the end.
strace	Pauses the program at each call, prints it, then lets it continue.	It can slow the program 10 times or more, so use it briefly on production.
Kernel	Performs the real file, network or memory work.	Its error code is the answer you are looking for.
Trace output	Calls with arguments, results and timing.	Millions of lines hide the cause; filter with -e trace.

Flow: Your program to strace / strace to Kernel / Kernel to strace / strace to Trace output

Useful options

- `-f` follows child processes, essential for shells and servers that fork.
- `-tt` adds timestamps; `-T` shows time spent in each call.

MARGA by EknathaLabs marga.eknathalabs.com

- `-e trace=file` or `network` or `process` limits the output to a group of calls.
- `-p PID` attaches to a running process; `-c` prints a summary table of calls and time.
- `-o trace.txt` writes to a file so the program's own output stays readable.

Reading a hang

A process that does nothing is usually blocked in one call. `strace -p PID` shows it: `futex(...)` waiting for another thread, `read(5, ...)` waiting for a socket, or `poll` waiting for events. File descriptor 5 can be matched to a socket with `ls -l /proc/PID/fd/5`.

Incident: The service that could not find its config

Incident: *A new unit file started the app from a different directory and it exited with no log line.*

```
$ systemctl status app
Active: failed (Result: exit-code)
$ strace -f -e trace=file ./app 2>&1 | tail -3
openat(AT_FDCWD, "config.yml", O_RDONLY) = -1 ENOENT (No such file or directory)
```

The app used a relative path and the unit file set no WorkingDirectory. Adding WorkingDirectory=/opt/app fixed it in one line. Without strace the team had been reading the application code for an hour.

Common mistakes

- Tracing without -f so the work in child processes is missed
- Reading the first lines of a huge trace instead of filtering
- Leaving strace attached to a busy production process
- Ignoring the error name and looking only at the call

Check yourself

What does ENOENT mean?

Answer: No such file or directory

Why use strace -f?

Answer: To follow child processes

A process is stuck. What helps most?

Answer: strace -p PID to see which call it is blocked in

Why avoid long strace runs in production?

Answer: It slows the traced process a lot

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 30. Seeing the network with ss and tcpdump

14 minutes. List sockets with ss and capture packets with tcpdump to prove whether traffic leaves, arrives and gets an answer.

When a service is unreachable, guessing between DNS, firewalls, routes and the application wastes time. Two commands split the problem quickly: ss tells you what the machine is listening on, and tcpdump tells you what really crosses its network interface.

What you will learn

- Reading ss output for listeners and connection states
- Writing a tcpdump filter for a host, port and TCP flags
- What a handshake, a refusal and a timeout look like in a capture
- Capturing safely and saving a file for later

Is it listening?

```
$ ss -ltnp
State   Recv-Q  Send-Q  Local Address:Port   Process
LISTEN  0       511     0.0.0.0:443          users:(("nginx",pid=812))
LISTEN  0       4096    127.0.0.1:5432       users:(("postgres",pid=640))
```

`0.0.0.0:443` accepts connections on every interface; `127.0.0.1:5432` only from this machine, so a remote client can never reach that database. Flags: `-l` listening, `-t` TCP, `-n` numbers instead of names, `-p` process. For connections use `ss -tn state established`, and watch `Recv-Q` or `Send-Q` growing, which means a slow reader or writer.

Do the packets arrive?

```
$ sudo tcpdump -ni eth0 'host 10.0.0.9 and tcp port 5432' -c 6
10.0.0.9.51022 > 10.0.0.5.5432: Flags [S], seq 1001
10.0.0.9.51022 > 10.0.0.5.5432: Flags [S], seq 1001      (retransmit)
10.0.0.9.51022 > 10.0.0.5.5432: Flags [S], seq 1001      (retransmit)
```

A normal handshake is `[S]`, then `[S.]` from the server, then `[.]`. Repeated `[S]` with no answer means the packet is dropped before the program sees it. `[R.]` back means the port is closed and the host refused. Many retransmissions of data packets means loss somewhere on the path.

Diagram: Splitting a connection problem

Part	What it does	What goes wrong
Client sends SYN	A connection attempt leaves the client.	If nothing leaves, the problem is on the client or its route.
tcpdump on server	Shows whether the SYN reaches the server interface.	No packet means the network path or a firewall upstream drops it.
Host firewall	Rules may drop or reject the packet before the service sees it.	A DROP rule gives silence; a REJECT gives a refusal.
ss listener	Shows the service bound to the right address and port.	Bound to 127.0.0.1 only, so remote clients are refused.
SYN-ACK back	The server answers and the handshake completes.	No reply with a listener present points at the firewall.

Flow: Client sends SYN to tcpdump on server / tcpdump on server to Host firewall / Host firewall to ss listener / ss listener to SYN-ACK back

MARGA by EknathaLabs marga.eknathalabs.com

Capturing safely

Always filter: a busy interface produces thousands of packets a second. Use `-c` to limit the count, and `-w file.pcap` to save a capture that you can open in Wireshark. Do not capture on a host that handles private data unless you are allowed to, because packets contain it. TLS traffic shows addresses and sizes but not content.

Incident: The database nobody could reach

Incident: *An application could not connect to a new database host, and everyone suspected the database.*

```
$ ss -ltn | grep 5432
LISTEN 0 4096 0.0.0.0:5432
$ tcpdump -ni eth0 port 5432
IP 10.0.0.9.51022 > 10.0.0.5.5432: Flags [S]   (6 times, no reply)
```

The database was listening and the SYN packets arrived, but nothing replied. A host firewall rule allowed only the old subnet. Adding the application subnet fixed the connection in a minute.

Common mistakes

- Capturing without a filter and drowning in packets
- Testing from the same machine and trusting that remote clients behave the same
- Reading a closed port and a dropped packet as the same failure
- Leaving a capture file with private data on a shared disk

Check yourself

What does a listener on 127.0.0.1 mean?

Answer: Only local connections can reach it

Repeated SYN packets with no reply suggest what?

Answer: The packet is dropped before the service answers

What does a RST reply mean?

Answer: The port is closed and the connection was refused

Why use -c and a filter with tcpdump?

Answer: To keep the capture small and relevant

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 31. Open files and /proc: find leaks and who holds what

13 minutes. Use lsof and the /proc filesystem to find which process holds a file, port or deleted file, and to spot descriptor leaks.

Linux treats files, sockets and pipes alike as open file descriptors. That is why one tool can answer very different questions: what is using this port, why will this disk not unmount, why does this process run out of handles. The /proc directory exposes the same facts for every process.

What you will learn

- How to ask who has a file, a port or a directory open
- How a deleted file keeps using disk space
- Reading limits and counts in /proc
- Spotting a descriptor leak

Who has it open?

```
$ sudo lsof -i :8080
COMMAND  PID  USER  FD  TYPE  NODE  NAME
java    2210  app   45u IPv6  TCP   *:8080 (LISTEN)
$ sudo lsof +D /var/log/app | head -3
$ sudo lsof +L1
COMMAND  PID USER  FD  SIZE/OFF   NLINK NAME
app     1820 app   4w  8589934592 0     /var/log/app/app.log (deleted)
```

`+L1` lists files with fewer than one link: **deleted but still open**. The disk space is freed only when the last process closes them. Truncating the file through the descriptor (`: > /proc/1820/fd/4`) or restarting the service releases it.

/proc for one process

Path	Shows
`/proc/PID/cmdline`	The full command line
`/proc/PID/fd/`	One link per open handle
`/proc/PID/limits`	Limits such as Max open files
`/proc/PID/status`	Memory, threads, state
`/proc/PID/cwd`	Working directory

Diagram: From a symptom to the process that holds it

Part	What it does	What goes wrong
Symptom	The disk is full, a port is busy or a process says too many open files.	Guessing which process is responsible wastes time.
lsof	Lists open files and sockets with the owning process.	Without sudo you only see your own processes.
/proc/PID	Shows handles, limits and memory for that process.	A process can change; read it more than once.
Fix	Restart, truncate, raise a limit or fix the leak.	Restarting hides a leak that returns tomorrow.
Watch the count	Count entries in /proc/PID/fd over time.	A number that only grows is a leak.

Flow: Symptom to lsof / lsof to /proc/PID / /proc/PID to Fix / /proc/PID to Watch the count

MARGA by EknathaLabs marga.eknathalabs.com

Spotting a leak

`ls /proc/PID/fd | wc -l` gives the number of open handles. Compare it with `Max open files` in `/proc/PID/limits`. If the count rises steadily under constant load and then the process fails with "Too many open files", code is opening sockets or files and not closing them. Raising the limit only delays the failure; find what is not closed.

Incident: The disk that was full but empty

Incident: *A disk showed 100 percent used while du reported half that.*

```
$ df -h /var
/dev/sdb1  100G  100G  0  100% /var
$ du -sh /var
52G  /var
$ lsof +L1
app  1820  app  4w  51539607552  0  /var/log/app/app.log (deleted)
```

Someone had removed the log with rm while the app kept writing to it. The space returned after the service restarted. The team switched to log rotation with copytruncate and an alert for large deleted-but-open files.

Common mistakes

- Deleting a big log file instead of truncating it while the process runs
- Restarting a leaking service every night and calling it fixed
- Raising ulimit without finding the leak
- Running lsof without sudo and seeing only half the story

Check yourself

df shows full but du shows half. What should you check?

Answer: Deleted files still held open by a process

Which command shows what listens on port 8080?

Answer: lsof -i :8080

Where do you read a process's open-file limit?

Answer: /proc/PID/limits

What does a steadily rising fd count mean?

Answer: A descriptor leak

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 32. Finding CPU, memory and I/O hogs

14 minutes. Use top, vmstat, iostat and pidstat to tell CPU, memory and disk problems apart and find the process responsible.

When a server feels slow, the first job is to find out what it is short of. Linux gives clear signals for CPU, memory and disk, and each has a standard command. Learning to read those signals in order turns a vague complaint into a named resource and a named process.

What you will learn

- Load average, CPU states and the run queue
- Memory, cache and swap: what is really used
- Disk latency and utilisation with iostat
- A short checklist for the first minute

The first minute

```
$ uptime
 14:02:11 up 41 days, load average: 9.80, 8.20, 5.10
$ nproc
4
$ vmstat 1 3
procs --memory-- --swap-- --io-- --cpu--
 r  b  swpd   free   si so  bi  bo  us sy id wa
12  3  81920  91000   0  0 220 900  20 10 10 60
```

Load average 9.8 on 4 cores means more runnable work than cores. In `vmstat`, `r` is the run queue, `b` processes blocked on I/O, `si/so` swap in and out, and `wa` the share of time the CPU waits for disk. Here `wa` is 60: the cores are waiting, not computing.

Which resource?

Signal	Points to
us + sy high, wa low	CPU bound
wa high, disk %util near 100	Disk bound
si/so above 0, little free memory	Memory pressure
Load high, CPU idle	Blocked on I/O or locks

```
$ iostat -x 1 2
Device  r/s   w/s  rkB/s  wkB/s  await  %util
sda     310   260  24000  18000   42.0   99.5
$ pidstat -d 1 1
UID  PID   kB_rd/s  kB_wr/s  Command
0    9001  21000    120      backup-tool
```

`await` is the average time a request waits, in milliseconds; `%util` near 100 means the device is saturated. `pidstat -d` names the process doing the I/O.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: The first-minute performance check

Part	What it does	What goes wrong
Load and CPU	uptime and vmstat show load, run queue and CPU states.	High load alone does not say which resource is short.
Memory and swap	free -m and vmstat si/so show pressure and swapping.	Cache is not used memory; look at available and swap activity.
Disk	iostat -x shows await and %util per device.	One saturated disk slows everything that touches it.
Process	top, pidstat or iotop names the process responsible.	Fixing the wrong process leaves the real cause.
Action	Move, throttle, schedule or fix the offender.	Adding hardware before finding the cause hides waste.

Flow: Load and CPU to Memory and swap / Memory and swap to Disk / Disk to Process / Process to Action

Memory is not what free first shows

Linux uses spare memory as cache, so "free" can look small on a healthy system. Read `available`. Real pressure shows as swap activity, the kernel killing processes (check `dmesg | grep -i oom`) or `si/so` above zero for long periods.

Incident: The CPU nobody could find

Incident: *A slow application showed low CPU and high load, and teams added cores that changed nothing.*

```
$ vmstat 1 2
r b swpd free  si so bi bo us sy id wa
12 3 81920 91000 0 0 220 900 20 10 10 60
$ iostat -x 1 1
sda  await 42.0  %util 99.5
```

The backup job shared a disk with the database. Moving it to off-peak hours and a separate volume restored normal latency, and a dashboard now shows I/O wait next to CPU.

Common mistakes

- Adding CPU before checking I/O wait
- Reading free memory as used memory and panicking
- Looking at average numbers only and missing short spikes
- Killing the top process without understanding what it does

Check yourself

Load average is high and CPU idle is high. What is likely?

Answer: Processes are blocked waiting, often for disk

Which column in vmstat shows CPU time waiting for disk?

Answer: wa

What does iostat %util near 100 mean?

Answer: The device is saturated

How do you find which process is writing to disk?

Answer: pidstat -d or iotop

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 33. Kernel messages, the journal and core dumps

13 minutes. Read dmesg and journalctl for kernel and service events, find out-of-memory kills, and collect a core dump after a crash.

The program's own log tells you what it knew. The kernel and the init system tell you what happened to it from outside: killed for memory, hit a segmentation fault, reset by a driver. Knowing these logs turns a mystery restart into a named cause.

What you will learn

- What dmesg shows and how to read the OOM killer
- Filtering the systemd journal by unit, priority and boot
- Making the journal survive a reboot
- Finding and opening a core dump

The kernel ring buffer

```
$ dmesg -T | grep -iE 'oom|killed|error' | tail -3
[Tue Nov  3 02:14:09 2026] Out of memory: Killed process 4411 (java) total-vm:9421112kB, anon-rss: \
    7884112kB
[Tue Nov  3 02:14:09 2026] oom_reaper: reaped process 4411 (java)
[Tue Nov  3 02:15:30 2026] EXT4-fs error (device sdb1): ext4_find_entry: reading directory
```

The **OOM killer** picks a process when memory runs out; the line names the victim and its memory. Disk and driver errors also appear here.

The journal

```
$ journalctl -u app.service --since "02:00" --until "02:30" -p warning
$ journalctl -b -1 -p err        # errors from the previous boot
$ journalctl -k                  # kernel messages only
$ sudo mkdir -p /var/log/journal && sudo systemctl restart systemd-journald
```

By default some systems keep the journal in memory only; creating `/var/log/journal` makes it persistent so you can read the boot before a crash.

Core dumps

When a program crashes with a signal such as SIGSEGV, the system can save a **core dump**. With systemd-coredump:

```
$ coredumpctl list
TIME                 PID  SIG  EXE
Tue 2026-11-03 02:14 4411 SEGV /usr/bin/app
$ coredumpctl gdb 4411
(gdb) bt
```

`bt` prints the stack at the moment of the crash, which a developer can match to code. Core files can contain secrets from memory, so store them with care and delete them when done.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: Where evidence of a crash lives

Part	What it does	What goes wrong
Process dies	Killed by the OOM killer, a signal or a driver fault.	The process cannot log its own sudden death.
dmesg	Kernel ring buffer with OOM, disk and driver messages.	It is a fixed-size buffer; old lines are overwritten.
journald	Collects service output and unit state changes.	Not persistent by default on some systems.
Core dump	A saved memory image for crashes by signal.	Large and may hold secrets; control access.
coredumpctl and gdb	Lists crashes and prints the stack.	Needs debug symbols for readable names.

Flow: Process dies to dmesg / dmesg to journald / journald to Core dump / Core dump to coredumpctl and gdb

Incident: The service killed by the kernel

Incident: *A Java service disappeared every few days with a clean application log.*

```
$ systemctl status app
Main PID: 4411 (code=killed, signal=KILL)
$ dmesg -T | grep -i killed
Out of memory: Killed process 4411 (java) anon-rss:7884112kB
```

The heap setting was larger than the memory limit of the host. They set the heap below the limit, added a memory alert, and made the journal persistent so the next incident would leave a trail.

Common mistakes

- Looking only in the application log for a killed process
- Leaving the journal in memory so a reboot erases the evidence
- Sharing a core dump without checking it for secrets
- Reading old dmesg lines after the buffer wrapped

Check yourself

Which log shows the OOM killer?

Answer: dmesg or the journal's kernel messages

How do you read errors from the previous boot?

Answer: journalctl -b -1 -p err

What does a core dump contain?

Answer: The memory image of a crashed process

Why make the journal persistent?

Answer: So logs survive a reboot

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 34. eBPF and bpftrace: ask live questions of a running system

14 minutes. Understand what eBPF is and use bpftrace one-liners and ready-made tools to answer live questions about files, processes and latency.

strace follows one program. eBPF lets you ask a question of the whole machine: who opens files, which processes start, how long do disk requests take. It is the modern way to observe Linux, and it is how many monitoring and networking tools work under the hood.

What you will learn

- What eBPF is and why the kernel can run it safely
- Probes: tracepoints, kprobes and uprobes
- Useful bpftrace one-liners
- Ready-made tools and their cost

What eBPF is

eBPF is a way to load small programs into the kernel. Before one runs, the kernel's **verifier** checks that it cannot crash or loop forever. The program attaches to an event, counts or times it, and stores results in a **map** that user space reads. No kernel module, no reboot, and no change to the application.

Probes

Probe	Fires when
`tracepoint:`	A stable kernel event happens, such as `syscalls:sys_enter_openat`
`kprobe:`	A kernel function is called
`uprobe:`	A function in a user program or library is called
`interval:`	Every N seconds, to print results

One-liners

```
$ sudo bpftrace -e 'tracepoint:syscalls:sys_enter_openat { @[comm] = count(); }'
^C
@[node]: 120
@[backup-tool]: 98213

$ sudo bpftrace -e 'tracepoint:sched:sched_process_exec { printf("%s -> %s\n", comm, str(args->fil \
    ename)); }'
$ sudo bpftrace -e 'kprobe:vfs_read { @bytes = hist(arg2); }'
```

The first counts file opens per process name: `backup-tool` stands out. The second prints every program that starts. The third shows a histogram of read sizes.

Ready-made tools from the BCC and bpftrace collections cover common questions: `execsnoop` (new processes), `opensnoop` (file opens), `biolatency` (disk latency histogram), `tcpconnect` (outgoing connections), `runqlat` (CPU scheduling delay).

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How an eBPF question is answered

Part	What it does	What goes wrong
Your question	For example: which process opens the most files.	A vague question gives a flood of output.
bpftrace script	Compiles the one-liner into a small kernel program.	A wrong probe name fails at load time, which is safe.
Kernel verifier	Rejects programs that could crash or loop forever.	Without it, tracing could take down the host.
Probe in the kernel	Runs on each matching event and updates a map.	A very hot probe adds overhead; keep programs small.
Results	The map is read and printed when you stop.	Remember to stop the trace; it keeps running until you do.

Flow: Your question to bpftrace script / bpftrace script to Kernel verifier / Kernel verifier to Probe in the kernel / Probe in the kernel to Results

Care and limits

eBPF needs a fairly recent kernel and root (or specific capabilities). Overhead is usually small, but a probe on a very busy function adds up, so stop traces when finished and test on a quiet host first. Output names a process by `comm`, which is only 15 characters; use PID when names collide. In containers, you see the host's processes unless you filter by cgroup.

Incident: The process nobody could name

Incident: *A host spent most of its time reading tiny files, and top showed many processes at low use each.*

```
$ sudo bpftrace -e 'tracepoint:syscalls:sys_enter_openat { @[comm] = count(); }'
@[backup-tool]: 98213
@[node]: 120
@[sshd]: 14
```

The backup tool scanned a directory with hundreds of thousands of small files every minute. The team changed it to run hourly with a list of changed files, and file-open counts dropped to normal.

Common mistakes

- Leaving a trace running after the question is answered
- Tracing a hot kernel function on a busy production host without testing
- Trusting comm names when two programs share a name
- Using eBPF when a simple log line would answer the question

Check yourself

What does the verifier do?

Answer: Checks an eBPF program is safe before it runs

Which probe type attaches to a stable kernel event?

Answer: tracepoint

When is eBPF better than strace?

Answer: When you do not know which process to look at

What is one risk of eBPF tracing?

Answer: Overhead on very busy probes

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 35. Debugging a slow or hung server: a method

14 minutes. Combine the earlier tools into a repeatable method: define the symptom, find the saturated resource, find the process and confirm the cause.

You now have tools for system calls, the network, open files, resources and the kernel log. The skill is the order in which you use them. A method keeps you from jumping to a favourite tool, and from changing three things at once so you cannot tell which one helped.

What you will learn

- Stating the symptom precisely before touching anything
- The USE walk: utilisation, saturation and errors per resource
- Narrowing from resource to process to call
- Changing one thing at a time and writing it down

1. State the symptom

"Slow" is not a symptom. Write: *the checkout page takes 8 seconds since 14:05; the API is fine; one of three servers is affected.* Add when it started, how many users are affected and what changed (a deploy, a config change, a traffic spike).

2. Walk the resources (USE)

For each resource ask three questions: **U**tilisation (how busy), **S**aturation (is work waiting), **E**rrors (anything failing).

Resource	Utilisation	Saturation	Errors
CPU	`top`, `mpstat`	Run queue (`vmstat r`)	Throttling, `dmesg`
Memory	`free -m`	Swap activity, OOM	OOM kills
Disk	`iostat %util`	`await`, queue size	`dmesg`, SMART
Network	`sar -n DEV`	Drops, retransmits (`ss -ti`)	Interface errors (`ip -s link`)

3. Narrow to a process, then a call

`top` or `pidstat` names the process. `strace -p` shows its blocked call, `lsof -p` its files and sockets, `ss -tnp` its connections, `journalctl -u` its log. If the host looks fine, check dependencies: DNS, a remote database, an NFS mount.

4. One change, then measure

Form a single hypothesis, change one thing, measure the same numbers again and record the result. If the numbers do not move, undo the change.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: The debugging loop

Part	What it does	What goes wrong
State the symptom	Who, what, since when and what changed.	A vague symptom sends you to the wrong tool.
USE walk	Check CPU, memory, disk and network for use, saturation and errors.	Skipping a resource leaves the real cause unseen.
Find the process	top, pidstat, ss and lsof name who is responsible.	Blaming the busiest process can hide the one that is blocked.
Find the call or line	strace, logs and traces show what it waits for.	Heavy tools on production can add load.
Change one thing	Apply one fix, then re-measure the same numbers.	Several changes at once teach you nothing.

Flow: State the symptom to USE walk / USE walk to Find the process / Find the process to Find the call or line / Find the call or line to Change one thing / Change one thing to USE walk

Write it down

Keep a short log: time, what you ran, what you saw, what you changed. It helps the incident channel, the postmortem and you in six months.

Incident: The one-hour guessing game

> **Incident:** *A team restarted services and resized the instance while the real problem was a hung NFS mount.*

```
$ ps -eo stat,comm | grep -c '^D'
41
$ dmesg -T | tail -2
nfs: server files.internal not responding, still trying
load average: 38.0, 35.2, 30.1
```

Processes in uninterruptible sleep (state D) were all waiting on the unreachable NFS server, so load was high and CPU idle. Fixing the file server restored the host. The team added the NFS mount to their dependency checklist.

Common mistakes

- Changing several settings at once
- Starting with a restart that destroys the evidence
- Skipping the symptom statement and chasing the loudest graph
- Not recording the steps taken

Check yourself

What does the USE method check per resource?

Answer: Utilisation, saturation and errors

Why change only one thing at a time?

Answer: So you know which change had the effect

High load, idle CPU and many D-state processes suggest what?

Answer: Processes blocked on I/O such as a hung mount

What should you write while debugging?

Answer: Time, command, result and change made

MARGA by EknathaLabs marga.eknathalabs.com

Cheat sheet: Linux

Files, permissions, processes, disks, logs and performance commands you reach for daily.

Navigate and inspect

Command	What it does	
`pwd`	Print the current directory	
`ls -lah`	Long list, hidden files, human sizes	
`cd -`	Return to the previous directory	
`tree -L 2`	Show a directory tree two levels deep	
`stat file`	Size, owner, permissions and timestamps	
`file name`	Detect the type of a file	
`` `du -sh * \ ``	sort -h`	Size of each item here, smallest to largest

Find

```
find /var/log -type f -mtime -1 -size +1M   # modified in the last day, larger than 1 MB
find . -name "*.conf" -not -path "./tmp/*"  # by name, excluding a path
grep -rn "error" /etc/nginx                  # search file contents recursively
type -a python3                              # which executable runs
```

Copy, move, link

```
cp -a src/ dst/              # copy keeping permissions and links
mv old new                   # rename or move
ln -sfn target linkname      # replace a symlink to a directory safely
readlink -f linkname         # resolve the final path
```

Users and permissions

Command	What it does
`id`	Your user, group IDs and groups
`sudo -l`	What you may run with sudo
`chmod 640 file`	Owner read/write, group read, others nothing
`chmod u+x script.sh`	Add execute for the owner
`chown app:app file`	Change owner and group
`umask`	Default permission mask for new files

Permission digits: read = 4, write = 2, execute = 1. Add them per class (owner, group, others), so `755` is `rwxr-xr-x`.

```
useradd -m -s /bin/bash deploy       # create a user with a home directory
usermod -aG docker deploy            # add to a group (keep existing ones)
passwd -l olduser                    # lock an account
getent passwd deploy                 # look up through all name sources
```

Processes and services

```
ps aux --sort=-%mem | head           # top memory users
ps -eo pid,ppid,stat,cmd | grep defunct   # find zombies
top -o %CPU                          # live view sorted by CPU
kill -TERM 1234                      # polite stop
kill -KILL 1234                      # last resort, cannot be caught
pgrep -af nginx                      # find by name with full command

systemctl status nginx               # state and recent log lines
systemctl enable --now nginx         # start now and at boot
systemctl restart nginx
systemctl list-units --failed        # everything that failed
systemctl daemon-reload              # after editing a unit file
```

MARGA by EknathaLabs marga.eknathalabs.com

Disks and filesystems

```
df -hT                                # space per filesystem with type
df -i                                 # inode usage (full inodes also stop writes)
lsblk -f                              # block devices and filesystems
findmnt /data                         # where and how a path is mounted
du -xh / --max-depth=1 2>/dev/null | sort -h | tail
lsof +L1                              # deleted files still held open
```

Logs

```
journalctl -u nginx --since "1 hour ago"
journalctl -p err -b                  # errors since boot
journalctl -f -u myapp                # follow live
tail -f /var/log/syslog
dmesg -T | tail                       # kernel messages with readable time
```

Performance triage (60 seconds)

```
uptime                                # load averages
vmstat 1 5                            # run queue, swap, I/O wait
free -h                               # memory and swap
iostat -xz 1 3                        # disk utilization and latency
ss -s                                 # socket summary
sar -n DEV 1 3                        # network throughput per interface
```

Symptom	Look at
Load high, CPU idle	I/O wait (`vmstat`, `iostat`)
Memory full, swap busy	`ps aux --sort=-%mem`, OOM messages in `dmesg`
"No space left" but `df -h` fine	`df -i` for inodes
Disk full after deleting	`lsof +L1` for open deleted files
Service flapping	`journalctl -u NAME`, `systemctl status NAME`

Text and data tools

```
grep -rn --include='*.conf' 'listen' /etc/nginx
grep -c 'ERROR' app.log
grep -v '^#' /etc/ssh/sshd_config | sed '/^$/d'
awk -F: '$3 >= 1000 {print $1, $3}' /etc/passwd
awk '{s+=$NF} END {print s}' sizes.txt
sed -n '20,40p' file.txt
sed 's/old/new/g' file.txt
cut -d, -f1,3 data.csv
sort data.txt | uniq -c | sort -rn | head
tr -s ' ' < file.txt
wc -l *.log
diff -u old.conf new.conf
xargs -a hosts.txt -P4 -I{} ping -c1 {}
tee -a out.log < input.txt
```

> **Danger:** `sed -i` *edits the file in place with no undo. Keep a copy first:* `sed -i.bak 's/a/b/' file`. *A mistyped* `xargs` *with* `rm` *can delete the wrong files, so run it with* `echo` *first.*

Archives and transfer

```
tar -czf backup.tar.gz /srv/app
tar -tzf backup.tar.gz | head
tar -xzf backup.tar.gz -C /restore
zip -r site.zip site/
unzip -l site.zip
rsync -avh --dry-run src/ user@host:/dst/
rsync -avh --delete src/ user@host:/dst/
scp file.txt user@host:/tmp/
curl -fsSLO https://example.com/file.tgz
sha256sum file.tgz
```

> **Danger:** `rsync --delete` *removes files at the destination that are missing from the source. Always run it with* `--dry-run` *first, and mind the trailing slash on the source:* `src/` *copies the contents,* `src` *copies the folder.*

MARGA by EknathaLabs marga.eknathalabs.com

Warning: `tar -xzf` *can overwrite existing files. Extract into an empty folder with* `-C`.

Permissions and ownership, more

```
stat -c '%a %U:%G %n' /etc/passwd
chmod u+x script.sh
chmod -R g+rX /srv/shared
chown -R app:app /srv/app
setfacl -m u:alice:rx /srv/data
getfacl /srv/data
umask
sudo -l
lsattr file
chattr +i /etc/important.conf
```

Warning: `chmod -R 777` *and* `chown -R` *on a wrong path (for example* `/` *or* `/etc`*) can break the system. Check the path with* `ls` *first. Never use 777 as a fix for "permission denied".*

Warning: `chattr +i` *makes a file immutable, even for root. Remember it before you wonder why edits fail.*

Network checks on a host

```
ip -br addr
ip route
ss -tulpn
ss -s
ping -c3 1.1.1.1
dig +short example.com
curl -sS -o /dev/null -w '%{http_code} %{time_total}s\n' https://example.com
traceroute -n example.com
nc -zv host 443
sudo tcpdump -i eth0 -nn port 53 -c 20
```

Warning: `tcpdump` *on a busy host can fill the disk and add load. Always limit it with* `-c`*, a filter and a time limit, and do not capture personal data you are not allowed to keep.*

Cautions for risky commands

Danger: `rm -rf` *has no recycle bin. Never run it with an unquoted variable such as* `rm -rf $DIR/*`*: if* `DIR` *is empty it becomes* `rm -rf /*`*. Use* `rm -rf -- "${DIR:?}"/`.

Warning: `dd of=/dev/sdX` *overwrites a disk with no confirmation. Confirm the device with* `lsblk` *and read the command twice.*

Danger: `mkfs`, `fdisk` *and* `parted` *on a mounted or wrong disk destroy data. Check* `lsblk -f` *and* `findmnt` *first.*

Warning: `kill -9` *gives a process no chance to clean up. Try* `kill` *(SIGTERM) first and wait.*

Warning: `reboot`, `shutdown` *and* `init 0` *over SSH on a remote server can leave you locked out. Confirm that you are on the right host with* `hostname`.

Warning: *Editing* `/etc/fstab`, `/etc/sudoers` *or* `sshd_config` *with a mistake can stop boot or lock you out. Use* `visudo`*, keep a second session open, and test with* `sshd -t` *before you reload.*

Warning: `iptables -F` *and changing the default policy over SSH can cut your own connection. Use a timed rollback such as* `at now + 5 minutes`.

MARGA by EknathaLabs marga.eknathalabs.com